Hands-On Machine Learning

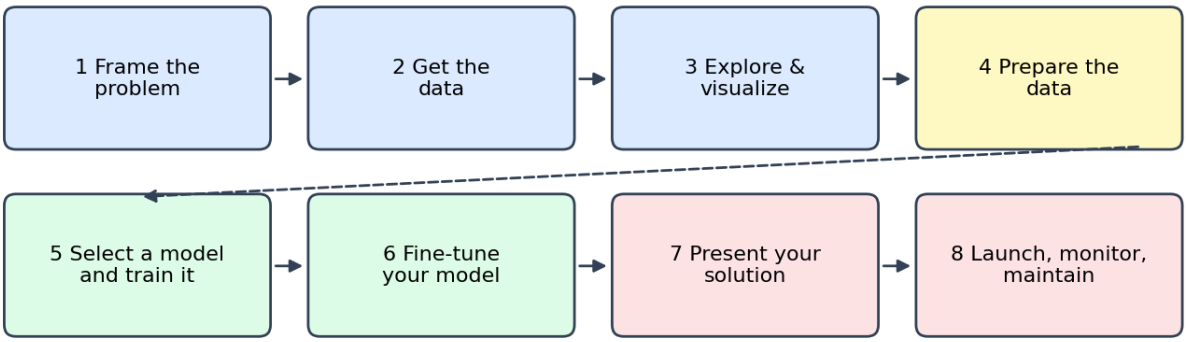
Chapter 2 — End-to-End Machine Learning Project

California Housing Prices

ملخص كشكولك + تكملة النقط الناقصة + رسومات وأكواد للمراجعة

من كشكولك | إضافة لإكمال النقص | انتبه / تصحيح

كل الأرقام في مربعات **Output (real run)** الخضرا طلعت من تشغيل الكود فعلًا على ملف housing.csv (20,640 منطقة). الأرقام المكتوب عليها «من الكتاب» هي أرقام الطبعة الثالثة: نسخة الداتا اللي اتحمّلت هنا بتدّي أرقام قريبة مش مطابقة 100%.



الخطوات الـ 8 لأي مشروع ML. الشابتر ده بيمشي عليهم كلهم بالترتيب.

1 الصورة الكبيرة وخطوات مشروع الـ ML

من كشكولك

الشابتر ده بيشتغّل معاك على مشروع من البداية للنهاية، والهدف إنه يوضحلك الخطوات الرئيسية لأي مشروع ML. الداتا اللي بنشتغل عليها هي California Housing Prices؛ مش حديثة بس فيها مميزات تعليمية كتير فالكتاب اعتبرها داتا تعليمية، وأضاف عليها عمود تصنيفي (`ocean_proximity`) وشال بعض الخصائص.

أنا مش هكتب كل كود في الكتاب، هكتب الكود والمصطلحات والأشياء اللي أول مرة أشوفها.

إضافة

تأطير المشكلة (Frame the problem) في المشروع ده:

- Supervised learning: عندنا labels (السعر الحقيقي لكل منطقة).
- Regression: بنتنبأ برقم (`median_house_value`) مش بفئة.
- Multiple regression: بنستخدم أكتر من feature للتنبؤ. وUnivariate: بنتنبأ بقيمة واحدة لكل منطقة.
- Batch learning: مفيش داتا داخلة باستمرار والداتا بتتحمّل على الذاكرة كلها.
- الداتا فيها **20,640** منطقة (district) في الملف ده.

الخطوات 7 و 8 (Present / Launch) مش موجودة في كشكولك، فهي متلخصة في آخر الملف (قسم 17).

2 نظرة على الداتا

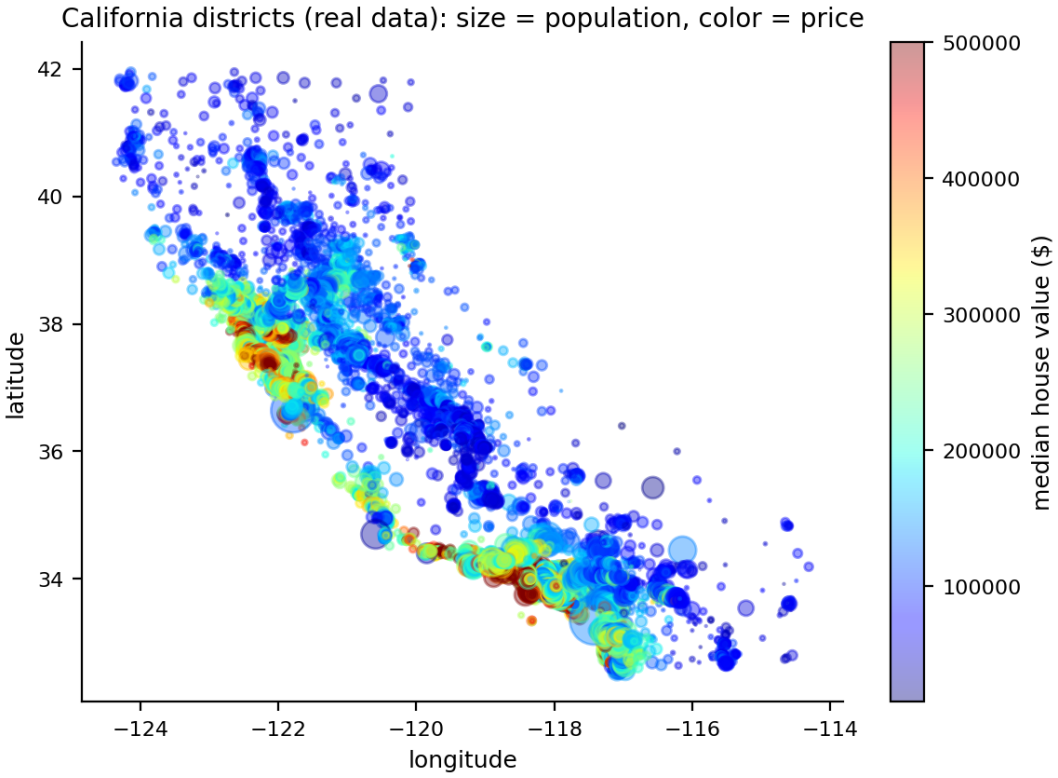
إضافة

قبل أي حاجة نحمّل الداتا ونبص عليها (الخطوة 2 و 3 في المشروع):

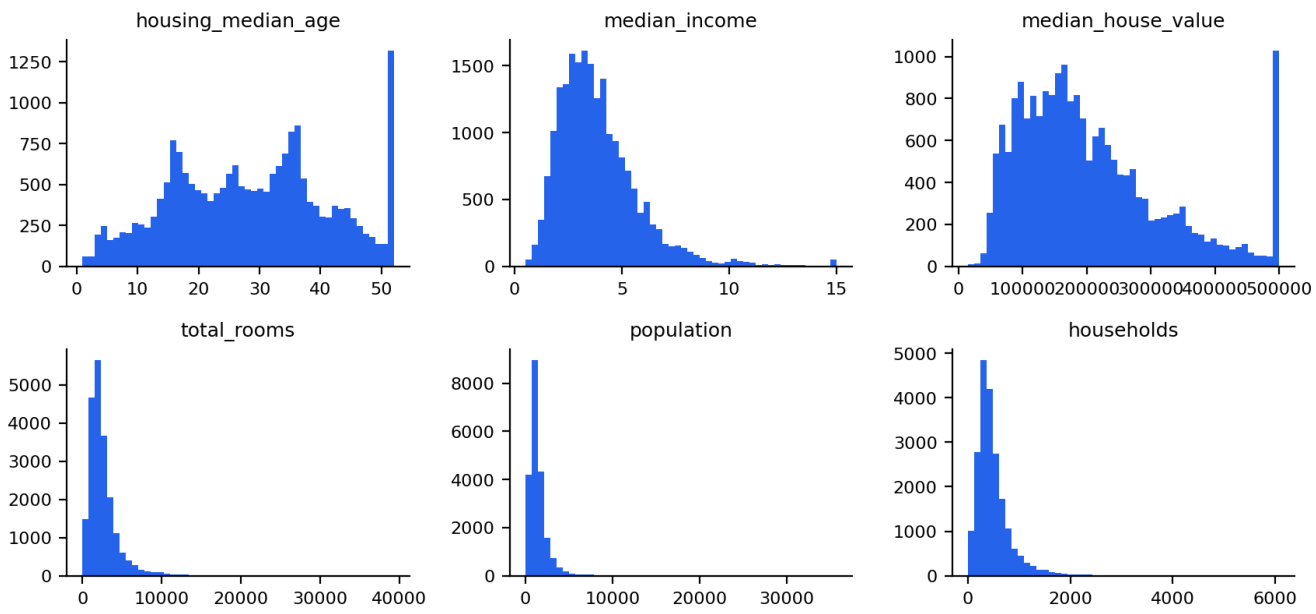
```
import pandas as pd
housing_full = pd.read_csv("housing.csv")
print(housing_full.shape)
print(housing_full["ocean_proximity"].value_counts())
print(housing_full["total_bedrooms"].isna().sum(), "missing values in total_bedrooms")
```

Output (real run)

```
(20640, 10)
<1H OCEAN   9136
INLAND      6551
NEAR OCEAN  2658
NEAR BAY    2290
ISLAND      5
207 missing values in total_bedrooms
```



الموقع الجغرافي (داتا حقيقية): كل نقطة منطقة، حجمها = السكان، ولونها = السعر. الأغلى (أحمر) قريب من المحيط والمدن الكبيرة. ده بيوضح إن الموقع feature مهم جدًا.

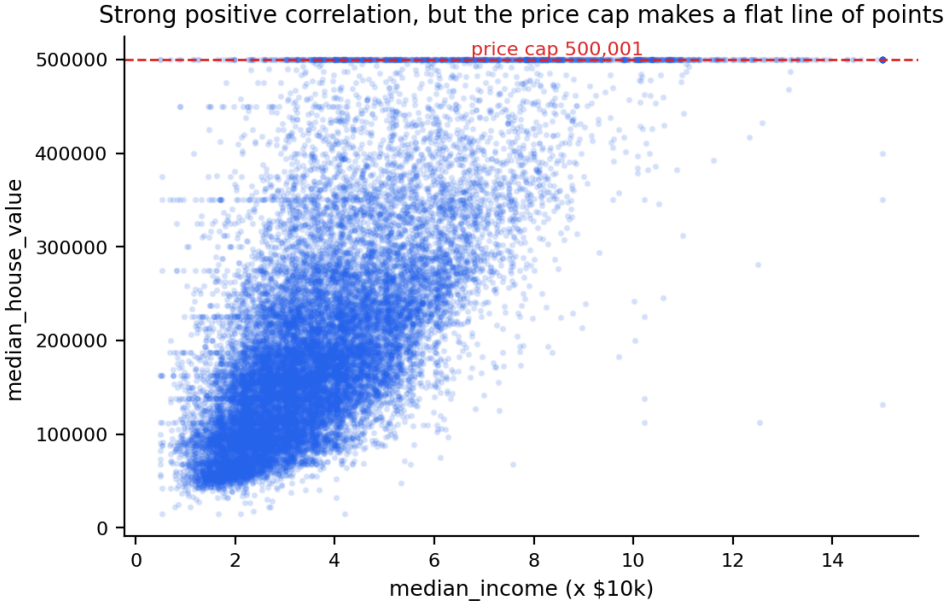


هيستوجرام لـ 6 أعمدة (حقيقي). لاحظ: `housing_median_age` مقصوص عند 52، و`median_house_value` مقصوص عند 500,001، والـ features الباقية "ذيلها طويل" لليمين.

إضافة

- **965** منطقة سعرها بالظبط 500,001 (السعر اتقص). ده خطر: لو الموديل اتدرّب على أسعار مقصوصة عمره ما هيتنبأ بسعر أعلى من 500 ألف.
- **1273** منطقة عمر بيوتها 52 (اتقص برضه).
- الـ `median_income` مش بالدولار: هي رقم بين 0.5 و 15 بالتقريب (بالعشرات آلاف).

ملحوظة من الكتاب: لو العميل محتاج أسعار فوق الـ 500 ألف لازم إما تجمع labels حقيقية للمناطق المقصوصة، أو تشيلها من الـ training والـ test.



الارتباط بين الدخل والسعر قوي، وفيه خط أفقي واضح عند 500,001 (السعر المقصوص).

3 مقاييس الأداء (RMSE و MSE و MAE)

من كشكولك

الأول لازم أختار Performance measure. المقياس المعتاد في مشاكل الـ Regression هو `RMSE` (Root Mean Square Error)؛ بيحسب الفرق العددي بين القيمة الفعلية والقيمة المتنبأ بيها.

$$\text{RMSE}(\mathbf{X}, h) = \sqrt{\frac{1}{m}\sum_{i=1}^{m}(h(\mathbf{x}^{(i)}) - y^{(i)})^2}$$

الرمز	معناه (من كشكولك)
`m`	عدد الحالات في مجموعة البيانات اللي بنقيس عليها (number of instances) في `RMSE`.
`i`	مجرد index، مش أُس.
`h`	الـ hypothesis، يعني الموديل نفسه (الدالة اللي بتتنبأ).
`x(i)`	vector فيه كل قيم الـ features للحالة رقم i (من غير الـ label)، زي: longitude, latitude, عدد الغرف... إلخ.
`y(i)`	القيمة الفعلية (Label) للحالة i، رقم واحد.
`h(x(i))`	اللي الموديل بيتنبأ بيه للحالة i، ونكتبه كمان `ŷ(i)`.

من كشكولك

الـ `RMSE` مخصوص لمشاكل الـ Regression بس. مش بنستخدمه في الـ Classification لأن النتيجة هناك فئة (Category) مش رقم، فبنستخدم Accuracy أو Precision / Recall / F1-score (خصوصًا لو في class imbalance زي كشف الاحتيال) أو Confusion matrix.

الـ `MSE` برضه مخصوص للـ Regression وبيستخدم نفس المنطق، وأغلب الموديلات بتستخدمه كـ loss function. الفرق إن `MSE` بيدي نفس معلومة الخطأ لكن مربعة (وحدتها دولار²)، فالـ `RMSE` مفهوم أكتر.

$$\text{MSE}(\mathbf{X}, h) = \frac{1}{m}\sum_{i=1}^{m}(h(\mathbf{x}^{(i)}) - y^{(i)})^2$$

على الرغم إن `RMSE` هو المقياس المفضل في الـ Regression، في بعض السياقات ممكن تفضّل مقياس تاني. مثلًا لو عندك كتير من الـ outliers ممكن تفكر في `MAE` (Mean Absolute Error)، وبيتسمّى كمان average absolute deviation.

$$\text{MAE}(\mathbf{X}, h) = \frac{1}{m}\sum_{i=1}^{m}|h(\mathbf{x}^{(i)}) - y^{(i)}|$$

من كشكولك

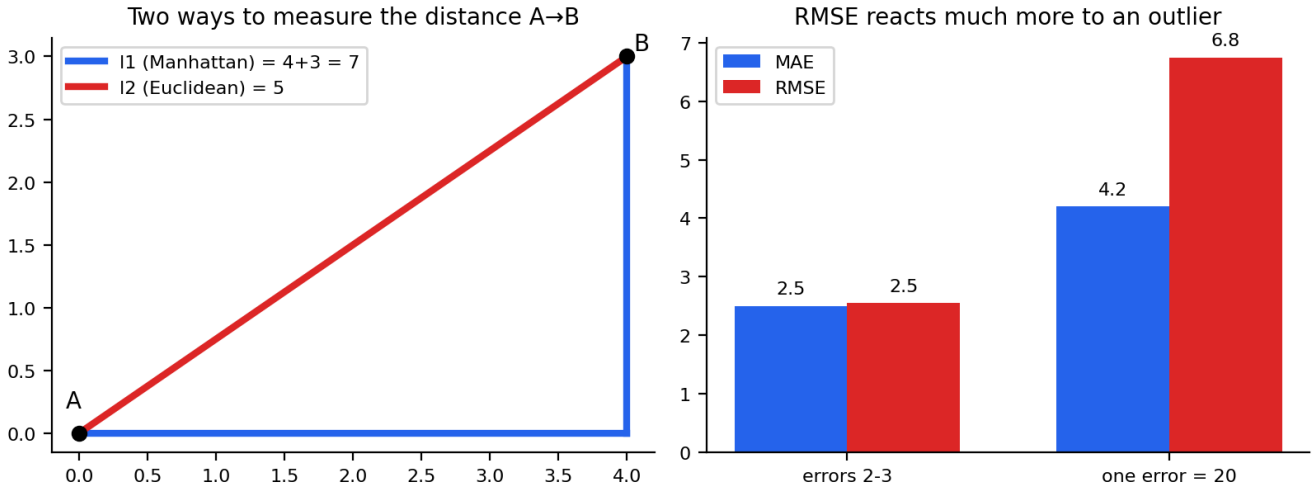
`RMSE` و `MAE` الاتنين بيقيسوا المسافة بين متجهين (vectors): متجه التنبؤات ومتجه القيم الحقيقية. في أكتر من طريقة لتعريف "المسافة" وده اللي بيسموه norms:

- **norm (Euclidean norm)** `ℓ2` ← `RMSE` بيستخدمها. نفس مفهوم المسافة المستقيمة اللي متعودين عليه (فيثاغورس).

• **norm (Manhattan norm)** ← `MAE` بيستخدمها. اسمها كده لأنها زي ما تتحرك في شوارع مدينة مربعات: مينفعش تمشي قطري، لازم تمشي بالطول والعرض. بتجمع القيم المطلقة من غير تربيع أو جذر. `ℓ1`

أهم ملاحظة: كل ما زاد رقم الـ norm، كل ما ركّز المقياس على القيم الكبيرة وتجاهل الصغيرة. عشان كده `RMSE` أكتر حساسية للـ outliers من `MAE`.

القاعدة: لو الـ outliers نادرة جدًا (زي منحنى الجرس) فالـ `RMSE` بيشتغل كويس جدًا. لو عندك outliers كتير استخدم `MAE`.



شمال: نفس المسافة بالـ ℓ2 والـ ℓ1. يمين (محسوب من الأخطاء 2-3 وخطأ واحد = 20): MAE طلع 2.5 → 4.2، لكن RMSE طلع 2.5 → 6.8.

إضافة

التعميم: الـ norm بالرقم `k` بتتحسب كده، وبنحصل على `ℓ1` و `ℓ2` بتغيير k:

$$\|\mathbf{v}\|_k = \left(|v_1|^k + |v_2|^k + \cdots + |v_n|^k\right)^{1/k}$$

$$\ell_2 : \ \|\mathbf{v}\|_2 = \sqrt{v_1^2 + v_2^2 + \cdots + v_n^2}$$

$$\ell_1 : \ \|\mathbf{v}\|_1 = |v_1| + |v_2| + \cdots + |v_n|$$

حالات خاصة: `ℓ0` بتعدّ عدد العناصر غير الصفرية، و `ℓ∞` بتدّيك أكبر قيمة مطلقة في الـ vector.

```
from sklearn.metrics import root_mean_squared_error, mean_absolute_error
rmse = root_mean_squared_error(y_true, y_pred)
mae  = mean_absolute_error(y_true, y_pred)
```

انتبه / تصحيح

في النسخ الحديثة من scikit-learn الـ parameter `squared=False` اتشال (اتعمله deprecation في 1.4 واتشال في 1.6). والبديل المباشر هو `root_mean_squared_error`. كود الكتاب (طبعته) بيستخدم `squared=False`، فلو شغّلته وطلع Error استخدم البديل. التجربة الفعلية على النسخة اللي عندي (1.8):

```
mean_squared_error([1, 2], [1, 3], squared=False)
```

Output (real run)

```
TypeError: got an unexpected keyword argument 'squared'
```

4 مقياس R^2 Score

من كشكولك

الـ R^2 score مقياس لتقييم موديلات الـ Regression. وظيفته الأساسية إنه يقولك: الموديل قدر يفسّر كام من التغيّر الموجود في القيم الحقيقية للـ Target، مقارنة بالتنبؤ بالمتوسط بس.

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

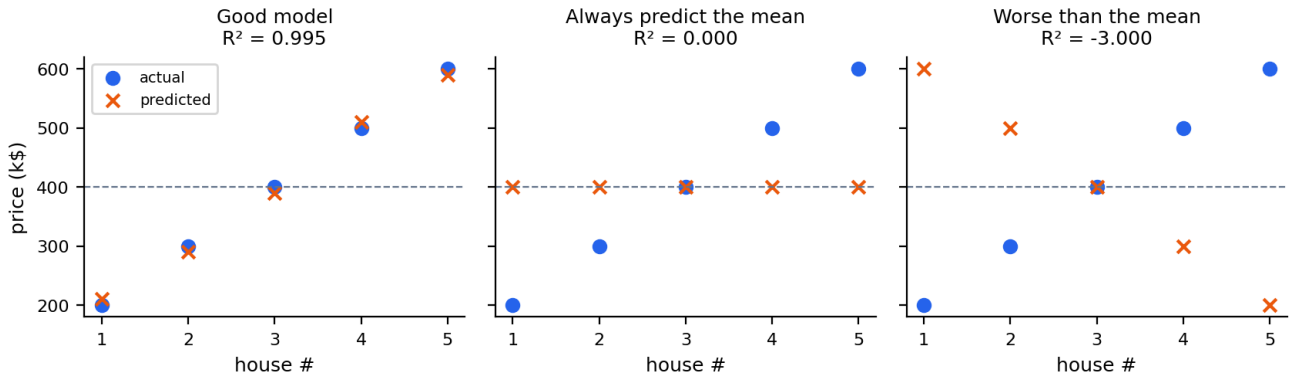
مثال: عندنا أسعار بيوت 200k, 300k, 400k, 500k, 600k. لو معملناش ML أصلًا وقلنا "أنا مش عارف سعر أي بيت، هتوقع متوسط الأسعار وخلاص" (المتوسط هنا = 400k) فكل بيت هنقول سعره 400k. ده بيتعتبر Baseline. الموديل بيستخدم الـ features (age, location, bedrooms) عشان يحاول يعرف أسعار البيوت المختلفة. بدل ما يقول 400k للكل، ممكن يقول: 210k, 290k, 390k, 510k, 590k. فالسؤال: الموديل قدر يفسّر كام من الاختلاف الموجود في أسعار البيوت؟ وده تقريبًا اللي بيقيسه R^2.

- **R^2 = 0**: الموديل مش قادر يفسر أي تباين؛ مش أفضل من إنك تستخدم متوسط الـ Target.
- **R^2 = 1**: الموديل بيفسر كل التباين بشكل مثالي؛ التنبؤات مطابقة للقيم الحقيقية (الحالة المثالية).
- **R^2 = 0.8**: الموديل بيشرح حوالي 80% من التباين في الـ Target مقارنة بالـ mean baseline. ده **مش** معناه إن الموديل بيتوقع صح بنسبة 80%، ولا إن الـ Accuracy = 80%، ولا إن 20% من التنبؤات غلط.

```
from sklearn.metrics import r2_score
r2_score([200, 300, 400, 500, 600], [210, 290, 390, 510, 590])
```

Output (real run)

```
0.995
```



نفس بيوتك: التنبؤات 210/290/390/510/590 تدّي R^2 = 0.995 (محسوب فعلًا). التنبؤ بالمتوسط بس = 0. وتنبؤ عكسي بيدّي قيمة سالبة (-3).

إضافة

الـ R^2 ممكن يطلع **سالب** لو الموديل أسوأ من التنبؤ بالمتوسط (زي الرسمة). الكسر في المعادلة = (مجموع مربعات أخطاء الموديل) ÷ (مجموع مربعات أخطاء الـ baseline بتاع المتوسط)، فكل ما الكسر صغر كل ما R^2 قرب لـ 1.

R^2 مقياس نسبي بلا وحدة، وRMSE بالوحدة الأصلية (دولار). الاتنين مكملين لبعض؛ R^2 يقولك "الموديل أحسن من الـ baseline بكام"، وRMSE يقولك "الخطأ النموذجي بكام دولار".

انتبه / تصحيح

اسم R^2 جاي أصلًا من (squared) لأنه في الـ linear regression البسيط بيساوي مربع معامل الارتباط r بين الـ x والـ y. المعادلة نفسها (1 − نسبة مجموعي المربعات) هي طريقة حسابه العامة.

5 إنشاء الـ Test Set

من كشكولك

الفكرة بسيطة: ناخد جزء عشوائي من الداتا (غالبًا 20%) ونحطه على جنب ونسميه test set. ده كود بيعمل كده يدويًا:

```
import numpy as np
def split_train_test(data, test_ratio):
    shuffled_indices = np.random.permutation(len(data))
    test_set_size = int(len(data) * test_ratio)
    test_indices = shuffled_indices[:test_set_size]
    train_indices = shuffled_indices[test_set_size:]
    return data.iloc[train_indices], data.iloc[test_indices]
```

من كشكولك

- `test_ratio` : حجم الداتا اللي هتروح للـ test (مثلًا 0.2).
- `np.random.permutation(len(data))` : بترجع أرقام الصفوف كلها بس مخلوطة عشوائيًا.
- `test_set_size` : عدد صفوف الـ test (الداتا × النسبة).
- `shuffled_indices[:test_set_size]` : أول جزء بالحجم ده للـ test، والباقي `[test_set_size:]` للـ train.
- `data.iloc[...]` : بترجع جدول جديد فيه الصفوف اللي في المواضع دي (بالخلطة، مش مترتبة).

أو ممكن أقول (جاهزة لأي داتا):

```
from sklearn.model_selection import train_test_split
train_set, test_set = train_test_split(housing_full, test_size=0.2, random_state=42)
print(len(train_set), len(test_set))
```

Output (real run)

```
16512 4128
```

إضافة

ليه نعزل الـ test set بدري؟ لو بصّيت عليه وإنت بتختار الموديل ممكن مخك يلتقط pattern فيه فيتحيّز اختيارك، وده بيتسمى data snooping bias، والنتيجة إن تقديرك للأداء الحقيقي يبقى متفائل زيادة.

`random_state=42` بتثبّت العشوائية: كل مرة تشغّل الكود هتاخد نفس التقسيم (الرقم نفسه مش مهم، 42 مجرد عُرف).

مشكلة تحديث الداتا

من كشكولك

لو قسّمت الداتا عشوائيًا ودرّبت وقيّمت، وبعد أسبوع جت داتا جديدة وضفتها لنفس الـ dataset ورجعت شغلت نفس كود التقسيم، تحصل مشكلة: حتى لو ثبّت `random_state` فـ `len(data)` نفسه اتغيّر، فالترتيب هيختلف تمامًا. يعني بعض الصفوف اللي كانت في الـ train ممكن دلوقتي تروح للـ test (والعكس). ودي مشكلة خطيرة لأن الموديل **شاف** بعض بيانات الـ test أثناء التدريب (data leakage) وده بيبوّظ مصداقية التقييم.

الحل: بدل ما أعتمد على "الترتيب العشوائي للفهرس" أعتمد على **هوية (ID)** كل صف: كل صف له ID ثابت، وأقرر إنه يروح للـ test بناءً على الـ ID نفسه مش على ترتيبه في الملف. فحتى لو الداتا زادت أو اتغير ترتيبها، الصف هيروح لنفس المجموعة.

إزاي أقرر؟ بـ hash function زي `crc32`: بتاخد أي مُدخل (رقم/نص) وترجع رقم ثابت. نفس المدخل يدي نفس الناتج دايمًا (عكس العشوائية)، والرقم الناتج موزع بشكل شبه عشوائي على مدى واسع من الأرقام. يعني `crc32(5)` هيرجّع دايمًا نفس الرقم مهما شغلت الكود كام مرة، وده الفرق الجوهري عن `np.random` اللي بيتغير كل مرة.

$$\text{crc32(id)} < \text{test_ratio} \times 2^{32} \Rightarrow \text{test set}$$

```
from zlib import crc32

def is_id_in_test_set(identifier, test_ratio):
    return crc32(np.int64(identifier)) < test_ratio * 2**32

def split_data_with_id_hash(data, test_ratio, id_column):
    ids = data[id_column]
    in_test_set = ids.apply(lambda id_: is_id_in_test_set(id_, test_ratio))
    return data.loc[~in_test_set], data.loc[in_test_set]

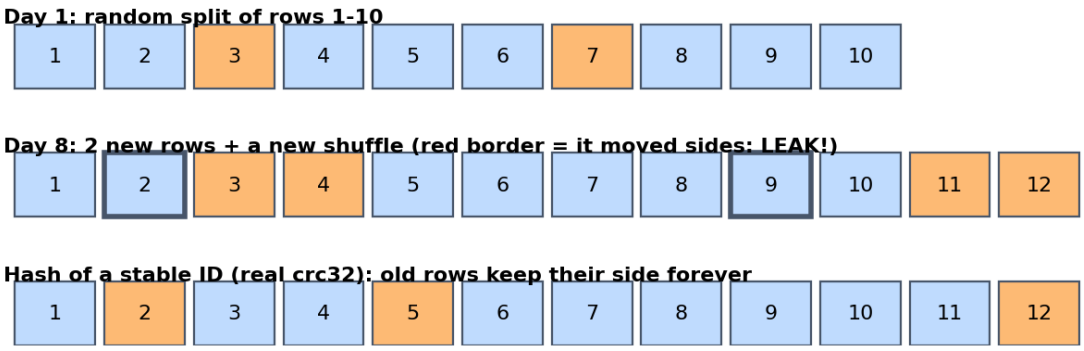
print([is_id_in_test_set(i, 0.2) for i in range(1, 11)])
```

Output (real run)

```
[False, True, False, False, True, False, False, False, False, False]   # ids 2 and 5 go to the test set - every single run
```

من كشكولك

الدالة الأولى بتاخد هوية الصف وتحسب الـ hash بتاعه (رقم بين 0 و 2^{32})، ولو أقل من `test_ratio × 2**32` (يعني أول 20% من المدى) الصف يروح للـ test (`True`) وإلا (`False`). الدالة التانية بتطبّق ده على كل صف: اللي `True` للـ test، واللي `False` (`~in_test_set`) للـ train.



التقسيم العشوائي المتكرر بيخلّي صفوف تتنقل بين train و test (الحدود الحمرا = تسريب)، لكن الـ hash بتاع ID ثابت بيحافظ على المجموعة. الصف الأخير من crc32 حقيقي: 2 IDs و5 و12 في الـ test.

إضافة

النسبة مش هتطلع بالظبط 20% في عينة صغيرة (هنا 2 من 10 صدفة طلعت 20%)، لكن مع داتا كبيرة هتقرب جدًا منها.

من كشكولك

دلوقتي الدالة `split_data_with_id_hash` محتاجة عمود هوية، و`housing` مفيهوش.

الحل الأول: استخدم رقم الصف نفسه كهوية:

```
housing_with_id = housing_full.reset_index()     # adds an "index" column
train_set, test_set = split_data_with_id_hash(housing_with_id, 0.2, "index")
```

من كشكولك

`reset_index()` بتاخد الـ index الداخلي بتاع الـ DataFrame وتحوّله لعمود عادي اسمه `index`.

مشكلة الحل ده: رقم الصف مش "هوية حقيقية" للبيت، هو بس ترتيبه في الجدول. لو مسحت صف من النص (مش من الآخر) كل الصفوف اللي بعده هيتغيّر رقمها (يطلع رقم واحد لفوق)، فالـ ID بتاعها يتغيّر رغم إن الحاجة نفسها ما اتغيّرتش، وده بيهد الفكرة كلها. (لازم الداتا الجديدة تتضاف في الآخر ومفيش صف يتمسح.)

الحل التاني: استخدم صفوف من الداتا نفسها لتكوين هوية (على سبيل المثال الموقع الجغرافي):

```
housing_with_id["id"] = housing_full["longitude"] * 1000 + housing_full["latitude"]
train_set, test_set = split_data_with_id_hash(housing_with_id, 0.2, "id")
```

من كشكولك

المهم الفكرة: عمود جديد اسمه `id` بيتكوّن من خصائص ثابتة ومميزة لكل صف.

انتبه / تصحيح

الحل التاني فيه فخ حقيقي جربته: الموقع الجغرافي مش فريد! على الداتا دي لقيت **8,050** صف (39% من الداتا) بيشاركوا نفس الـ id مع صف تاني (مناطق كتير بنفس الإحداثيات بالظبط). ده معناه إن المجموعات دي هتروح كلها مع بعض للـ train أو للـ test، فالتقسيم بيبقى "بالمجموعات" مش بالصفوف، وده بيقلل عشوائية العينة شوية. لازم تعرف ده: الـ ID المثالي يكون **فريد وثابت**.

6 السحب الطبقي (Stratified Sampling)

من كشكولك

تخيّل صندوق فيه 10 كور: 6 حمرا و 4 زرقا (يعني النسبة 60% / 40%). المطلوب نسحب 4 كور بس من الصندوق (عينة صغيرة) وعايزين العينة دي تمثّل الصندوق كله.

- **الطريقة الأولى: سحب عشوائي بسيط.** ممكن الـ 4 كور اللي هتطلع متطلعش بنسبة 60/40 تقريبًا، وممكن بالصدفة يطلعوا كلهم حمرا (يعني القلق من الصدفة).
- **الطريقة التانية: Stratified Sampling (السحب الطبقي):** بدل ما نسحب عشوائي من الصندوق كله، نعمل: (1) نفصل الكور لمجموعتين: كومة حمرا وكومة زرقا. (2) عايز أسحب 4 كور من أصل 10 يعني 40% من الصندوق. (3) نطبّق نفس الـ 40% على كل كومة لوحدها.

الهدف: لما يكون عندي فئة مهمة في بياناتي، الـ Stratified بيضمن إن الـ split test/train ياخد كل فئة بنفس نسبتها الحقيقية، بدل ما السحب العشوائي يسيب الموضوع للصدفة اللي ممكن تطلّع عينة مش ممثّلة.

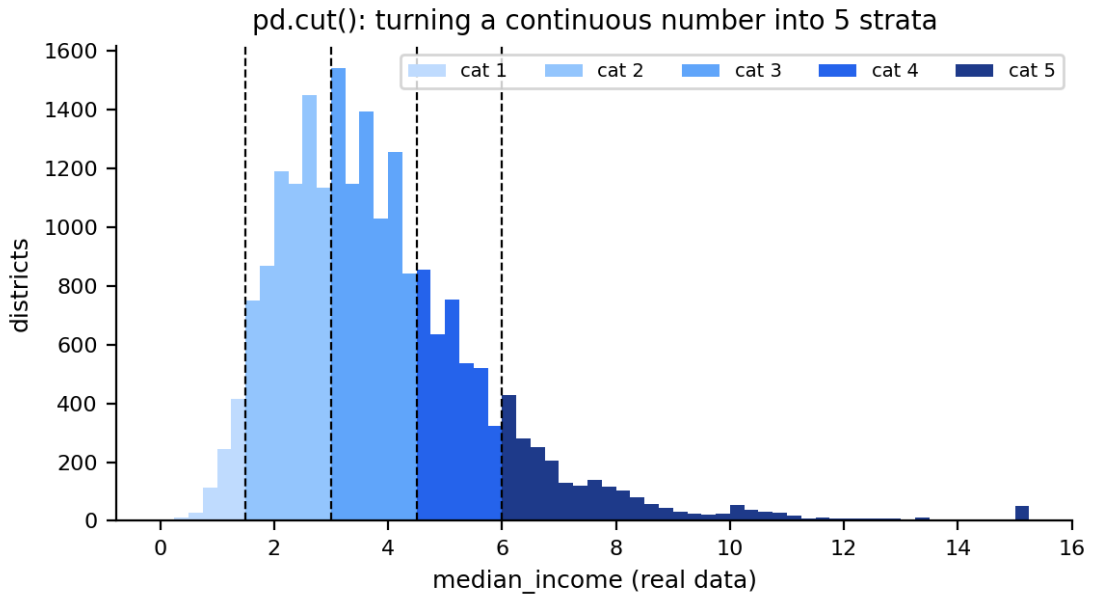
من كشكولك

لكن لو الـ feature رقم مستمر (زي `median_income`) مفيش "مجموعات" واضحة أقسم عليها. الحل: أعمل أنا مجموعات (strata) بنفسي بـ `pd.cut()`:

```
housing_full["income_cat"] = pd.cut(housing_full["median_income"],
                                    bins=[0., 1.5, 3.0, 4.5, 6., np.inf],
                                    labels=[1, 2, 3, 4, 5])
```

من كشكولك

`pd.cut()` بتاخد عمود رقمي مستمر وتقسّمه لفئات على حسب حدود (bins): من 0 لحد 1.5 ← فئة 1، من 1.5 لحد 3 ← فئة 2، وهكذا. والـ `labels` هي أسماء الفئات. النتيجة: عمود جديد `income_cat` وكل بيت بقى ليه "فئة" (من 1 لـ 5) بدل الرقم العشري. دي الـ strata.



الخطوط المتقطعة = حدود الـ bins، وكل لون = فئة (من الداتا الحقيقية).

من كشكولك

دلوقتي أقسّم الداتا مع الحفاظ على نسبة كل فئة. في طريقتين، واحدة كبيرة وواحدة سهلة وهي:

```
strat_train_set, strat_test_set = train_test_split(
    housing_full, test_size=0.2, stratify=housing_full["income_cat"], random_state=42)

for set_ in (strat_train_set, strat_test_set):      # remove income_cat so it is never used as a feature
    set_.drop("income_cat", axis=1, inplace=True)
```

من كشكولك

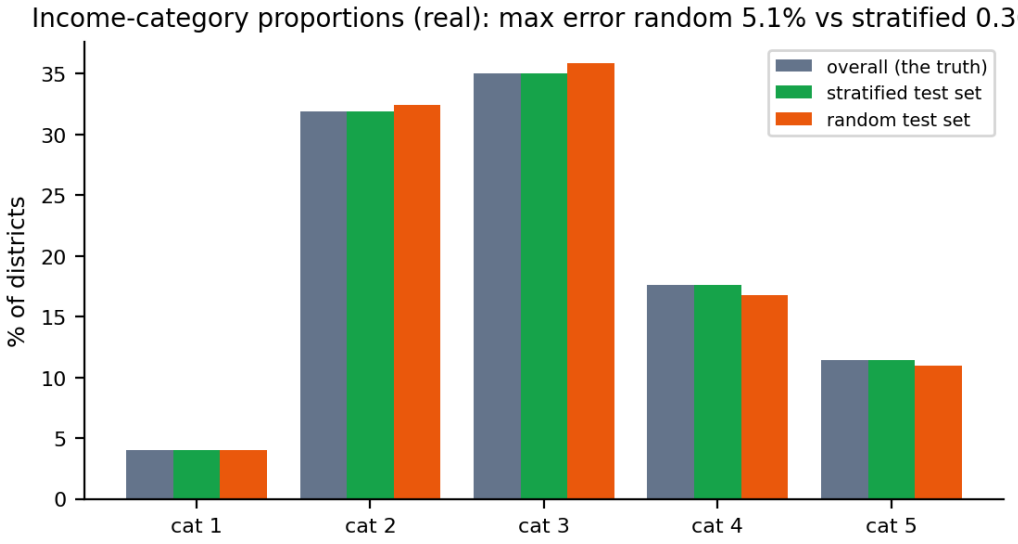
هي بالظبط `train_test_split` اللي قلنا عليها قبل كده بس بإضافة `stratify`. وبعد ما استخدمنا `income_cat` لازم أمسحه من الـ train والـ test عشان ما يدخلش غلط كـ feature في تدريب الموديل.

إضافة

نتيجة حقيقية: نسبة كل فئة دخل في الداتا كلها مقابل الـ test set (stratified vs random):

income_cat	% Overall	% Stratified	% Random
1	3.98	4.00	4.02
2	31.88	31.88	32.44
3	35.06	35.05	35.85
4	17.63	17.64	16.74
5	11.44	11.43	10.95

أكبر انحراف نسبي عن الحقيقة: العشوائي **5.1%** مقابل الـ stratified **0.36%**. ليه `income` بالذات؟ الكتاب بيقول إن الدخل المتوسط أهم عامل للتنبؤ بسعر البيت، فمهم إن الـ test set يمثّل كل فئات الدخل بنسبها الحقيقية.



الأعمدة الخضرا (stratified) ملزوقة في الحقيقة، والبرتقالي (random) بيزيغ شوية. (من تشغيل حقيقي)

7 هندسة الـ Features وتحليل الارتباط

من كشكولك

Feature engineering: استخلاص معلومات مفيدة عن طريق دمج أو تحويل أعمدة موجودة (زي القسمة بينهم) عشان أوصل لعمود جديد أكتر ارتباطًا وتدليلًا على الحاجة اللي بحاول أتنبأ بيها، بدل ما أعتمد على الأعمدة الخام لوحدها. وأتأكد إنها مفيدة عن طريق مقارنة الـ `corr` قبل وبعد.

```
housing["rooms_per_house"]  = housing["total_rooms"] / housing["households"]
housing["bedrooms_ratio"]   = housing["total_bedrooms"] / housing["total_rooms"]
housing["people_per_house"] = housing["population"] / housing["households"]

corr_matrix = housing.corr(numeric_only=True)
corr_matrix["median_house_value"].sort_values(ascending=False)
```

Output (real run)

```
median_house_value     1.0000
median_income          0.6872
rooms_per_house        0.1463
total_rooms            0.1351
housing_median_age     0.1141
households             0.0646
total_bedrooms         0.0478
people_per_house      -0.0220
population            -0.0269
longitude             -0.0475
latitude              -0.1427
bedrooms_ratio        -0.2600
```

إضافة

من الكتاب: `median_income` ≈ 0.688، `rooms_per_house` ≈ 0.144، و`bedrooms_ratio` ≈ −0.256. في الأرقام اللي فوق من تشغيلي: `median_income` = **0.687**، `rooms_per_house` = **0.146**، `bedrooms_ratio` = **-0.260** (سالب: كل ما نسبة غرف النوم للغرف الكلية قلّت، السعر بيميل يزيد). يعني `bedrooms_ratio` أكتر إفادة من `total_rooms` و`total_bedrooms` الخام.

تذكير: `corr()` بتقيس العلاقة **الخطية** بس؛ ممكن علاقة قوية غير خطية تطلع قيمتها صغيرة.

8 القيم الناقصة (SimpleImputer) والـ Outliers

من كشكولك

عشان أتعامل مع missing values في الداتا بعد الـ drop، أملأ مكان القيم الناقصة دي بقيمة مناسبة. `SimpleImputer` = "الأداة اللي بتملا القيم الناقصة". هو كلاس جاهز من sklearn بيعمل نفس فكرة `fillna()` بتاعت Pandas، بس بطريقة معيارية ومصممة تتكامل مع باقي أدوات sklearn.

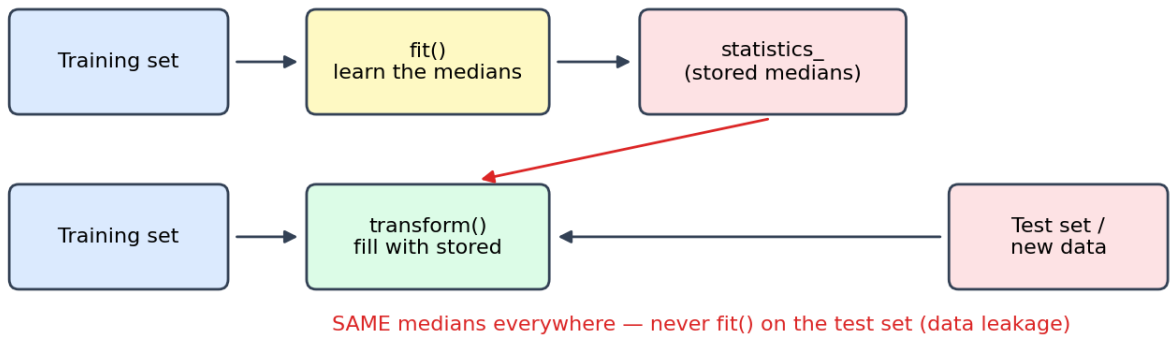
```
from sklearn.impute import SimpleImputer
imputer = SimpleImputer(strategy="median")
housing_num = housing.select_dtypes(include=[np.number])   # the median only works on numbers
imputer.fit(housing_num)
print(imputer.statistics_)          # the median learned for each column
X = imputer.transform(housing_num)   # really fills the missing values
```

```
Output (real run)
[-118.51   34.26   29.00 2119.00  433.00 1164.00  408.00    3.54]
```

من كشكولك

- `strategy="median"` : بتقول للأداة لما تلاقي قيمة ناقصة املاها بالوسيط (median) بتاع العمود؛ تمامًا مثل `housing["total_bedrooms"].fillna(median)` بس هنا بتحسب لكل الأعمدة.
- أنا استخدمت `SimpleImputer` بدل `fillna()` لأنها مصممة عشان تشتغل جوه الـ pipeline ويحفظ القيمة اللي اتعلّمها، فنقدر نطبّقها تلقائيًا على بيانات جديدة من غير ما نحسب الوسيط من جديد كل مرة.
- `fit()` : معناها "اتعلّم من البيانات دي". بيحسب الوسيط لكل عمود ويحفظه من غير ما يغيّر في الداتا نفسها.
- `transform()` : دلوقتي طبّق اللي اتعلّمته فعلًا على الداتا (ملء كل القيم بالوسيط المحفوظ).
- **ليه فصل بين fit و transform؟** لأن ده الأساس في فلسفة sklearn: `fit` على الـ training data بس (تتعلم الوسيط)، وبعدها `transform` على أي بيانات تانية (زي الـ test set) بنفس الوسيط المتعلَّم من الـ training، مش بنحسب وسيط جديد من الـ test (ده اسمه data leakage بين الـ train والـ test).
- **ملاحظة:** `transform()` بترجّع NumPy array، ولو عايز DataFrame:

```
housing_tr = pd.DataFrame(X, columns=housing_num.columns, index=housing_num.index)
```



fit على الـ training فقط، وبعدين transform لأي حاجة بنفس القيم المحفوظة.

إضافة

في الـ training set عندنا **158** قيمة ناقصة في `total_bedrooms` (الأعمدة التانية مفيهاش ناقص)، وSimpleImputer هيملاهم بالوسيط (433).

الـ strategies المتاحة في `SimpleImputer` : `"mean"` (الافتراضي)، `"median"` ، `"most_frequent"` (بتشتغل مع النصوص كمان)، `"constant"` (قيمة تحددها بنفسك بـ `fill_value`). هنستخدم `most_frequent` مع الأعمدة النصية لاحقًا في القسم 14.

كشف الـ Outliers بـ Isolation Forest

من كشكولك

خوارزمية بتكتشف الصفوف "الغريبة" تلقائيًا وبترجّع `-1` للصف الشاذ (outlier) و `1` للصف العادي.

```
from sklearn.ensemble import IsolationForest
isolation_forest = IsolationForest(random_state=42)
outlier_pred = isolation_forest.fit_predict(X)
print((outlier_pred == -1).sum(), "outliers out of", len(X))

# if you want to drop them after identifying them:
housing = housing.iloc[outlier_pred == 1]
housing_labels = housing_labels.iloc[outlier_pred == 1]
```

Output (real run)

```
2116 outliers out of 16512
```

9 ترميز الأعمدة النصية (Encoding)

من كشكولك

موديلات الـ ML خوارزميات رياضية بتفهم أرقام بس، مش نصوص (يعني لو عندك عمود زي `INLAND` لازم أحوّله لأرقام بطريقة ما).

الحل الأول: `OrdinalEncoder` — الفكرة: ياخد كل فئة نصية ويدّيها رقم (0, 1, 2, ...). `fit_transform()` اختصار لـ `fit` و `transform` مع بعض في سطر واحد.

```
from sklearn.preprocessing import OrdinalEncoder
ordinal_encoder = OrdinalEncoder()
housing_cat_encoded = ordinal_encoder.fit_transform(housing_cat)   # housing_cat = the text column
print(housing_cat_encoded[:5].ravel(), housing_cat.values[:5].ravel())
print(ordinal_encoder.categories_)
```

Output (real run)

```
[1.0 4.0 1.0 4.0 0.0] ['INLAND' 'NEAR OCEAN' 'INLAND' 'NEAR OCEAN' '<1H OCEAN']
[['<1H OCEAN' 'INLAND' 'ISLAND' 'NEAR BAY' 'NEAR OCEAN']]
```

من كشكولك

المشكلة: لما يدّي مثلًا `INLAND = 1` و `ISLAND = 2`، الموديل هيفهم إن 1 و 2 قريبين، و 1 و 4 بعيدين، وكأن في ترتيب منطقي حقيقي بين الفئات، وده بيفترض علاقة غلط بين الفئات.

من كشكولك

الحل الأصح: `OneHotEncoder` — الفكرة: بدل ما نديّ كل فئة رقم، بنصنع عمود منفصل لكل فئة، ونحط `1` في العمود اللي بيمثّل الفئة الصح و `0` في كل الباقي. كده الموديل مش بيفترض أي ترتيب أو قرب بين الفئات؛ كلهم متساويين ومنفصلين تمامًا.

```
from sklearn.preprocessing import OneHotEncoder
cat_encoder = OneHotEncoder()
housing_cat_1hot = cat_encoder.fit_transform(housing_cat)
print(housing_cat_1hot.shape, type(housing_cat_1hot).__name__)
print(housing_cat_1hot.toarray()[:3])
```

Output (real run)

```
(16512, 5) csr_matrix
[[0. 1. 0. 0. 0.]
 [0. 0. 0. 0. 1.]
 [0. 1. 0. 0. 0.]]
```

إضافة

الفئة	OrdinalEncoder	OneHotEncoder
`<1H OCEAN`	0	[0 ,0 ,0 ,0 ,1]
`INLAND`	1	[0 ,0 ,0 ,1 ,0]
`ISLAND`	2	[0 ,0 ,1 ,0 ,0]
`NEAR BAY`	3	[0 ,1 ,0 ,0 ,0]
`NEAR OCEAN`	4	[1 ,0 ,0 ,0 ,0]

مشكلة Ordinal: 0 و 1 "قريبين"، 0 و 4 "بعيدين" بدون سبب منطقي. وفي حالتنا `ocean_proximity` أصلًا مالهاش ترتيب حقيقي (باستثناء ISLAND اللي حاجة لوحدها).

`pd.get_dummies()` سهلة لكنها "مبتفتكرش" الفئات اللي شافتها، فلو ظهرت فئة جديدة في الداتا اللي بعد كده هتعمل أعمدة مختلفة. أما `OneHotEncoder` فبتحفظ الفئات في `fit()` وممكن تتحكم في الفئة الجديدة بـ `handle_unknown="ignore"` (بنستخدمها في الـ pipeline لاحقًا).

لو عدد الفئات كبير جدًا، الـ one-hot هيعمل عدد ضخم من الأعمدة؛ ساعتها بيتفكر في بدايل زي الـ embeddings (هتتشرح في الـ Deep Learning).

Sparse Matrix

من كشكولك

لما أجي أعرض `housing_cat_1hot` بلاقي حاجة غريبة: أغلب القيم في الجدول أصلًا أصفار، لأن كل صف فيه رقم `1` واحد بس من أصل 5 أعمدة. sklearn بترجّع الناتج كـ SciPy sparse matrix بدل NumPy array عادي، يعني بتخزّن بس مواقع الأرقام غير الصفرية (الـ 1 بس) مش كل الأصفار. ده بيوفّر ذاكرة جدًا (تخيل لو عندك 1000 فئة: كل صف هيخزن 999 صفر من غير فايدة).

```
housing_cat_1hot.toarray()                          # to see it as a normal array (with all the zeros)
cat_encoder = OneHotEncoder(sparse_output=False)      # or ask for a dense array from the start

cat_encoder.feature_names_in_                         # the names of the input columns
cat_encoder.get_feature_names_out()                   # the names of the new columns after encoding
```

مفيدة عشان تبني DataFrame واضح بأسماء أعمدة مفهومة بدل أرقام مجردة.

Output (real run)

```
['ocean_proximity_<1H OCEAN' 'ocean_proximity_INLAND'
 'ocean_proximity_ISLAND' 'ocean_proximity_NEAR BAY'
 'ocean_proximity_NEAR OCEAN']
```

10 توحيد المقياس (Feature Scaling)

من كشكولك

العملية اللي بتاخد كل الأعمدة (Features) عندك وتخليهم كلهم في نطاق متقارب أو موحّد، بدل ما كل عمود يكون له مقياس مختلف تمامًا عن التاني.

المشكلة: لو العمود الأول يتراوح مثلًا من 0 لـ 40 ألف (مدى واسع جدًا) والتاني من 0 لـ 20 (مدى صغير جدًا). معظم خوارزميات الموديلات بتحسب حسابات رياضية تعتمد على الاختلافات بين الأرقام. لو غذّيت الموديل بالأرقام دي زي ما هي، العمود الأول هيسيطر لأن أرقامه أكبر، مش لأنه أهم. فنحط كل الأعمدة في نفس "الملعب الرقمي"، والموديل بيقارن بين الأعمدة بعدالة.

الحل الأول: Min-Max Scaling

$$x_{\text{scaled}} = \frac{x - x_{\text{min}}}{x_{\text{max}} - x_{\text{min}}}$$

```
from sklearn.preprocessing import MinMaxScaler
min_max_scaler = MinMaxScaler(feature_range=(-1, 1))
housing_num_min_max_scaled = min_max_scaler.fit_transform(housing_num)
print(housing_num_min_max_scaled.min(axis=0).round(1), housing_num_min_max_scaled.max(axis=0).round(1))
```

Output (real run)

```
[-1.0 -1.0 -1.0 -1.0 -1.0 -1.0 -1.0 -1.0] [1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0]
```

من كشكولك

النتيجة: أصغر قيمة في العمود بتبقى دايمًا 0 (لأن `x - x_min = 0`)، وأكبر قيمة بتبقى 1 (لأن البسط = المقام)، وكل القيم التانية بتقع بينهم. أحيانًا بنغيّر المدى الافتراضي بدل (0, 1) ممكن نحدد مدى مختلف (زي (−1, 1)) لو الموديل بتاعنا بيفضّل قيم حوالين الصفر (زي الشبكات العصبية في الـ Deep Learning). المعادلة بتتعدل شوية عشان توصل للمدى الجديد، لكن الفكرة الأساسية نفسها.

الحل التاني: Standardization (أقوى مع الـ outliers)

$$x_{\text{scaled}} = \frac{x - \mu}{\sigma}$$

```
from sklearn.preprocessing import StandardScaler
std_scaler = StandardScaler()
housing_num_std_scaled = std_scaler.fit_transform(housing_num)
print(housing_num_std_scaled.mean(axis=0).round(2), housing_num_std_scaled.std(axis=0).round(2))
```

Output (real run)

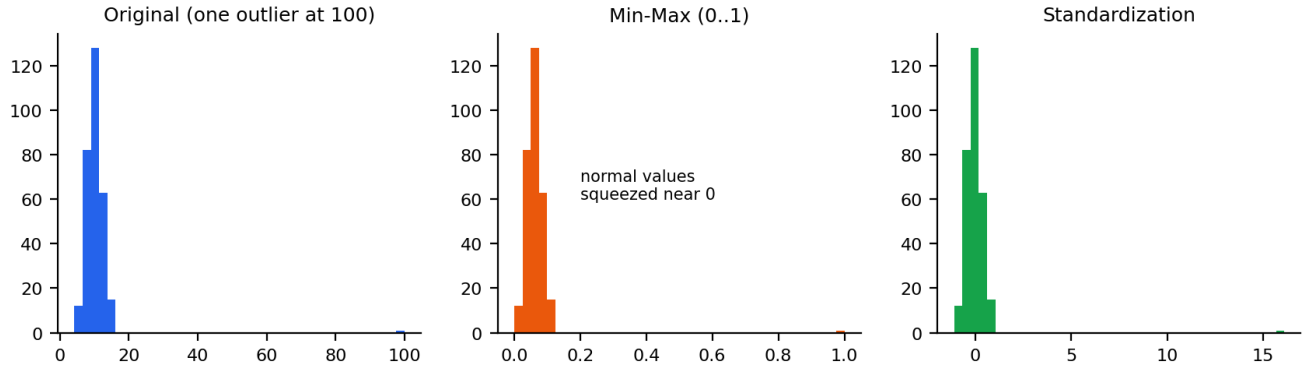
```
[ 0.00  0.00  0.00 -0.00 -0.00 -0.00 -0.00 -0.00] [1.00 1.00 1.00 1.00 1.00 1.00 1.00 1.00]
```

من كشكولك

يعني: (القيمة − متوسط العمود μ) ÷ (الانحراف المعياري σ). **الفرق الجوهري عن Min-Max:** هنا مفيش مدى ثابت (0 إلى 1)، لكن متوسط العمود بيبقى 0 والانحراف المعياري 1. والقيم ممكن تطلع أي رقم (موجب أو سالب)، مش محدودة بحد معين.

ليه أقوى مع الـ outliers؟ لو كل البيانات من 0 لـ 20 وحصلت غلطة إدخال وكتب 100 (أو 200). مع Min-Max القيمة الكبيرة دي هتبقى هي الـ max، فكل باقي القيم الطبيعية (0 إلى 20) هتتحوّل لقيم صغيرة جدًا قريبة من الصفر (لأن `x_max − x_min` بقى ضخم). أما في Standardization فالمتوسط والانحراف المعياري بيتأثروا شوية بس، فالنتيجة أحسن.

القاعدة نفسها زي SimpleImputer: أي Scaler لازم `fit()` على الـ training set بس، وبعدين `transform()` على الـ test set وأي بيانات جديدة بنفس الـ scaler. مش بنعمل `fit()` على الـ test set.



داتا فيها outlier واحد (100) (synthetic، تشغيل حقيقي): Min-Max ضغط القيم العادية بين 0.00 و 0.12 فقط، بينما Standardization حافظ على انتشار أحسن (-1.1 إلى 0.9).

إضافة

- **متى أستخدم أيهم؟** Min-Max لما محتاج مدى محدد (زي 0..1 أو −1..1، خصوصًا للشبكات العصبية). Standardization لما الداتا فيها outliers أو مش عايز مدى ثابت.
- الموديلات اللي بتعتمد على الأشجار (Decision Tree / Random Forest) عمومًا **مش حساسة** لمقياس الـ features، لكن الـ Linear models والـ KNN والشبكات العصبية بتستفيد جدًا من الـ scaling.

11 تحويلات الـ Features (Log و Bucketizing و RBF)

Log Transform

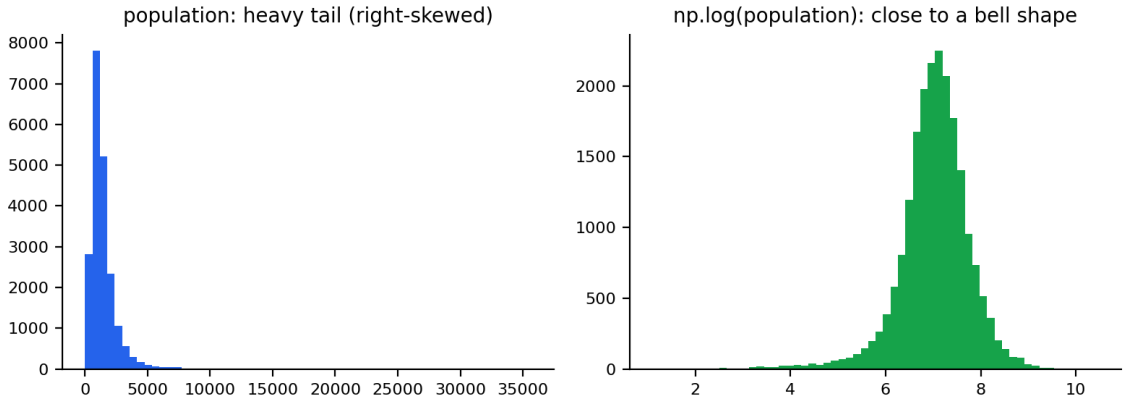
من كشكولك

المشكلة: بعض الـ features شكل توزيعها "heavy tail" (ذيل ثقيل)؛ يعني أغلب القيم متجمعة في نطاق صغير (زي معظم الأحياء فيها 1,000 إلى 3,000 نسمة) لكن في عدد قليل جدًا من الحالات الشاذة بأرقام ضخمة (10,000 أو 20,000 نسمة). فشكل التوزيع بيبان كذيل طويل ممتد ناحية اليمين (right-skewed). الموديلات بتشتغل أحسن لما شكل توزيع البيانات قريب من التوزيع الطبيعي (Gaussian / bell-shaped). لو العمود "heavy tail" الموديلات بتلاقي صعوبة تتعلم منه بشكل صحيح.

الحل: خد لوغاريتم العمود. اللوغاريتم بيضغط الفروقات الكبيرة (بين 10000 و 20000 مثلًا) وبيوسّع الفروقات الصغيرة، فالنتيجة النهائية بتقرّب جدًا من الشكل الجرسي المثالي.

```
import numpy as np
housing["population"] = np.log(housing["population"])     # the name of the column you want to take the
log of
print(housing_full["population"].skew(), np.log(housing_full["population"]).skew())
```

```
Output (real run)
4.936  -1.071      # skewness: +4.9 (long tail)  ->  -1.1 (almost symmetric)
```



توزيع population الحقيقي قبل وبعد اللوغاريتم.

إضافة

اللوغاريتم بيشتغل على قيم **موجبة بس** (log لصفر أو لرقم سالب مش معرّف). لو في أصفار استخدم `np.log1p`.

من الكتاب: لو الـ heavy tail ناحية اليمين، أول حاجة ممكن تجربها الجذر التربيعي (أو الرفع لأُس بين 0 و 1). واللوغاريتم بيتستخدم لما الـ tail طويل جدًا (power law) زي `population`. وفي المشروع اتطبّق الـ log على الأعمدة: `total_bedrooms, total_rooms, population, households, median_income`.

Bucketizing

من كشكولك

الفكرة الأساسية: بدل ما نسيب القيمة الأصلية (رقم مستمر)، نقسّم مدى القيم لـ "صناديق" (buckets / bins) متساوية الحجم تقريبًا، ونستبدل كل قيمة برقم الصندوق اللي وقعت فيه (مش القيمة نفسها). شبه الفكرة اللي استخدمتها في Stratified sampling مع `pd.cut`، لكن الهدف مختلف: هنا بنستخدمها على feature عشان نعطيها للموديل (التقسيم الطبقي كان للـ split).

مثال: لو عندي عمود يتراوح من 0 لـ 15 وقسّمته 5 صناديق متساوية (equal-sized buckets).

- **الحالة الأولى:** لما كل صندوق فيه تقريبًا نفس عدد الصفوف (زي أول 20% من القيم يروحوا صندوق 1، تاني 20% صندوق 2...) النتيجة إن العمود بقى شبه موزّع بانتظام (uniform) ومفيش حاجة اسمها scaling أصلًا لعمود زي ده.
- **الحالة التانية:** لما العمود يكون متعدد القمم (Multimodal) يعني شكل توزيعه فيه أكتر من قمة (modes)، مش شكل جرس واحد. مثال: عمر البيوت بيتجمع حوالين 20 سنة وتجمع تاني حوالين 35 سنة. الـ Bucketizing في الحالة دي بيخلّي رقم الصندوق نتعامل معاه كـ category (مش رقم عادي) ونستخدم `OneHotEncoder` عليه. فالموديل يتعامل مع كل صندوق كفئة منفصلة تمامًا، ويتعلم لكل فئة سلوك مختلف. ده الأفضل لأن في حالة التوزيع متعدد القمم

القرب الرقمي مش دايمًا يبقى مفيد (الصندوق 3 "قريب" لصندوق 4 رقميًا، لكنه ممكن يشبه صندوق 1 أكتر في السلوك).

إضافة

مثال الكتاب على الـ multimodal هو `housing_median_age` : في الداتا الحقيقية عندنا قمة غريبة عند 52 (1273 منطقة، ده الـ cap)، وقمم تانية حوالين 35 و36 (824 و 862 منطقة). ممكن البيوت اللي اتبنت من حوالي 35 سنة ليها طراز قديم خرج من الموضة، فأسعارها أرخص مما يوحي به عمرها لوحده. الـ OneHot على الـ buckets بيسمح للموديل يتعلم قاعدة مختلفة لكل مدى عمر. والـ RBF بيدّيك طريقة تانية: feature لكل قمة (mode) رئيسية.

RBF-Similarity Features

من كشكولك

الفكرة: تخيّل عايز تعرف "إيه مدى قرب رقم معين من رقم تاني ثابت؟" (مش هل هو أكبر أو أصغر). مثال: عايز تعرف قد إيه عمر البيت قريب من 35 سنة. لو العمر 35 فالقرب = 1 (أقصى درجة)، لو العمر 34 أو 36 فالقرب = 0.9، لو العمر 10 أو 60 فالقرب = 0.1 أو أقل (بعيد جدًا). ده بالظبط اللي RBF (Radial Basis Function) بتحسبه: مقياس "التشابه/القرب" بين قيمة معينة ونقطة ثابتة مهمة.

ليه نحتاج ده؟ زي ما قلنا لو لقينا إن في تجمّع كبير من البيوت عمرها حوالين 35 سنة وسعرها (مثلًا) رخيص، بدل ما نعمل bucketize (فئات منفصلة) ممكن نعمل حاجة أدق: نصنع feature جديد بيقول "قد إيه البيت قريب من عمر 35 تحديدًا" (يعني بدل عمود العمر العادي، يبقى عندنا عمود جديد اسمه "التشابه مع عمر 35" بيتراوح بين 0 (بعيد جدًا) و 1 (35 بالظبط)).

```
from sklearn.metrics.pairwise import rbf_kernel
age_simil_35 = rbf_kernel(housing[["housing_median_age"]], [[35]], gamma=0.1)

# on 5 sample ages (35, 34, 36, 10, 60):
print(rbf_kernel(pd.DataFrame({"housing_median_age": [35, 34, 36, 10, 60]}), [[35]],
gamma=0.1).ravel().round(3))
```

Output (real run)

```
[1.000 0.905 0.905 0.000 0.000]
```

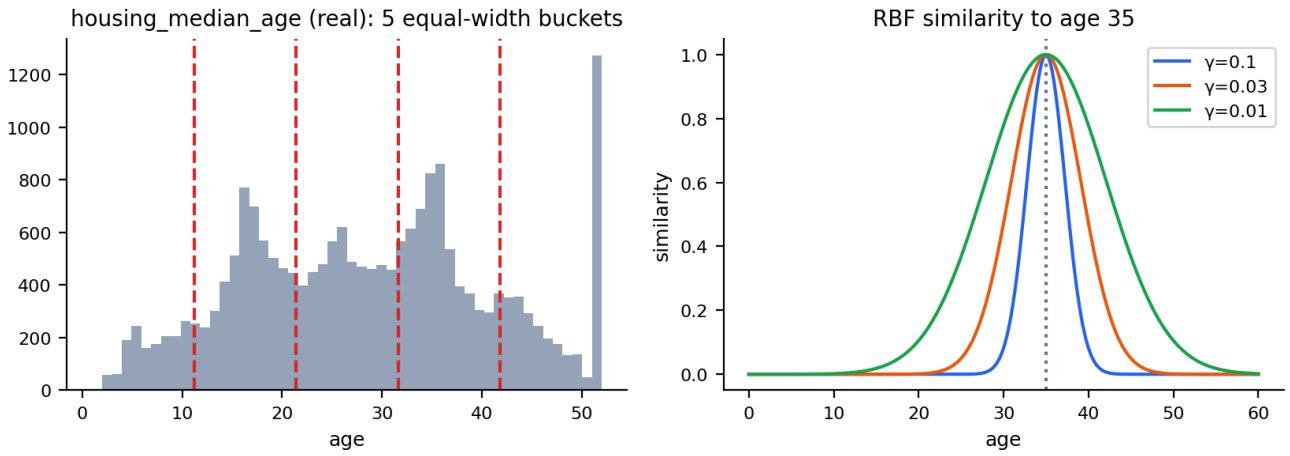
من كشكولك

`gamma` معامل بيتحكم في "سرعة" تناقص التشابه كل ما ابتعدنا عن 35.

المعادلة الرياضية:

$$\text{similarity} = \exp\left(-\gamma(x-35)^2\right)$$

دالة أُسّية `exp` بتاخد الرقم السالب الكبير وتحوّله لرقم قريب من الصفر، وكل ما الرقم اللي جوه `exp` قريب من صفر (يعني x قريب من 35) الناتج بيكون قريب من 1.



شمال: 5 buckets متساوية العرض على `housing_median_age` الحقيقي. يمين: منحنى RBF للتشابه مع 35 بقيم γ مختلفة (γ أكبر = أضيق وأسرع هبوط).

إضافة

من الكتاب: الـ RBF اللي بنقيس بيها التشابه مع 35 بتوصل لأقصى قيمة (1) عند 35، وده بالظبط جنب القمة (spike) الموجودة في توزيع `housing_median_age`. والـ RBF الأكثر استخدامًا هي الـ Gaussian RBF. ولاحظ إن النتيجة اللي فوق مطابقة للمعادلة: عمر 34 و 36 = 0.905 (مش 0.9 بالظبط: ده تقريب).

12 تحويل الـ Target والـ Custom Transformers

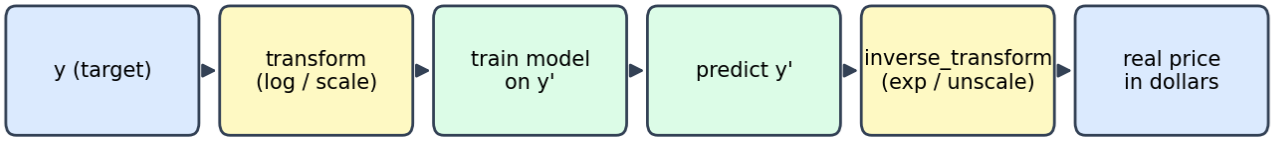
من كشكولك

الـ Bucketizing والـ RBF والـ Log transform والـ scaling كلها اتطبّقت على الـ features. دلوقتي الموضوع مختلف: بنتكلم عن تحويل الـ target نفسه (الـ label؛ القيمة اللي الموديل بيحاول يتنبأ بيها).

ليه أحوّل الـ target؟ لنفس سبب الـ features: لو توزيع الـ target نفسه "heavy tail" ممكن أحوّله (زي اللوغاريتم) عشان التدريب يبقى أسهل وأدق للموديل. لكن بتظهر مشكلة جديدة: لو أخدت لوغاريتم الـ target (مثلًا لوغاريتم median) الموديل هيتنبأ بلوغاريتم السعر مش بالسعر نفسه، فدلوقتي أحتاج أعمل inverse transform (الدالة العكسية، يعني الأُس `exp`) عشان أرجّع السعر الحقيقي بالدولار.

الحل: في طريقتين: الأولى يدوية والتانية أفضل وأدق:

```
from sklearn.compose import TransformedTargetRegressor
from sklearn.linear_model import LinearRegression
model = TransformedTargetRegressor(LinearRegression(), transformer=StandardScaler())
model.fit(housing[["median_income"]], housing_labels)
predictions = model.predict(some_new_data)
```



TransformedTargetRegressor بيحوّل الـ target للتدريب وبيرجّعه لقيمته الحقيقية تلقائيًا.

Custom Transformers

من كشكولك

`FunctionTransformer` : الفكرة بتاعتها: لو عاوز تعمل تحويل بسيط (زي log و/أو RBF).

المشكلة الحقيقية إن كل الأدوات اللي استخدمناها قبل كده (`StandardScaler` , `SimpleImputer` , `OneHotEncoder`) كلها عندها نفس الشكل: `fit()` و `transform()` و `fit_transform()` . ودي مش صدفة؛ عشان نقدر نربطهم ببعض في Pipeline. لو استخدمت `np.log()` مباشرة هي بترجع array بس وملهاش fit/transform، فمش هتتحط في Pipeline ولا هتتضم مع الأدوات التانية. `FunctionTransformer` بتغلّف أي function في transformer بنفس شكل أدوات sklearn، فتقدر تتحط في pipeline وتتركّب مع غيرها، والـ pipeline يشغّل الكل تلقائيًا بالترتيب.

```
from sklearn.preprocessing import FunctionTransformer
# Ex 1
log_transformer = FunctionTransformer(np.log, inverse_func=np.exp)
log_pop = log_transformer.transform(housing[["population"]])
# Ex 2
rbf_transformer = FunctionTransformer(rbf_kernel, kw_args=dict(Y=[[35.]], gamma=0.1))
age_simil_35 = rbf_transformer.transform(housing[["housing_median_age"]])
```

إضافة

لو التحويل محتاج يتعلّم حاجة من الداتا بجد (`fit()` حقيقية)، زي الـ mean في `StandardScaler` أو مراكز الـ clusters في `KMeans`، بتكتب class بتورث من `BaseEstimator` و `TransformerMixin` وتعرّف `fit()` و `transform()` . الكتاب بيعمل كده في `ClusterSimilarity` (RBF + KMeans)، وده الكود الكامل:

```
class ClusterSimilarity(BaseEstimator, TransformerMixin):
    def __init__(self, n_clusters=10, gamma=1.0, random_state=None):
        self.n_clusters = n_clusters; self.gamma = gamma; self.random_state = random_state
    def fit(self, X, y=None, sample_weight=None):
        self.kmeans_ = KMeans(self.n_clusters, n_init=10, random_state=self.random_state)
        self.kmeans_.fit(X, sample_weight=sample_weight)
        return self                                   # fit() must return self
    def transform(self, X):
        return rbf_kernel(X, self.kmeans_.cluster_centers_, gamma=self.gamma)
    def get_feature_names_out(self, names=None):
        return [f"Cluster {i} similarity" for i in range(self.n_clusters)]

cluster_simil = ClusterSimilarity(n_clusters=10, gamma=1., random_state=42)
similarities = cluster_simil.fit_transform(housing[["latitude", "longitude"]])
print(similarities.shape); print(similarities[0].round(2))
```

Output (real run)

```
(16512, 10)
[0.00 0.16 0.82 0.00 0.00 0.00 0.00 0.00 0.47 0.34]
```

الفكرة: KMeans بيلاقي 10 مراكز جغرافية (مراكز تجمعات سكانية)، وبعدين بنقيس تشابه كل منطقة مع كل مركز بـ RBF. النتيجة 10 features جديدة بتقول للموديل "المنطقة دي قريبة قد إيه من مركز كذا".

13 الـ Pipelines

من كشكولك

المشكلة: دلوقتي عندي خطوات متعددة (`SimpleImputer` ملء القيم الناقصة، `StandardScaler` قياس) كل واحدة منفصلة والزم أشغّلها واحدة واحدة وأمرر ناتج واحدة للتانية. لو عندك سلسلة طويلة من الخطوات (زي 5-6 خطوات) هتضطر تكتبهم واحدة واحدة وتتابع تسلسلهم بإيدك، وده عرضة للغلط ومش منظّم.

الحل: Pipeline

```
from sklearn.pipeline import Pipeline
num_pipeline = Pipeline([
    ("impute", SimpleImputer(strategy="median")),
    ("standardize", StandardScaler()),
])
```

من كشكولك

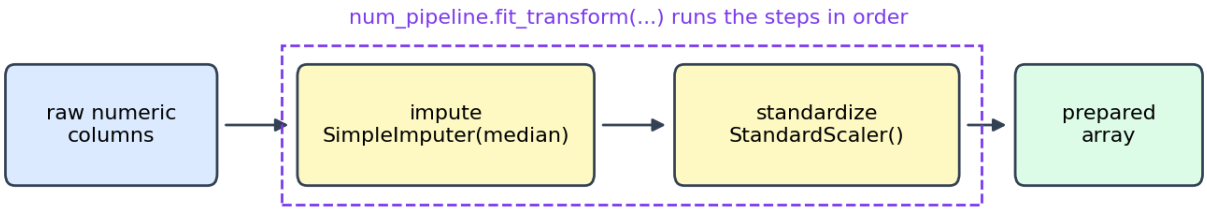
بياخد قائمة من الخطوات بالترتيب، كل خطوة عبارة عن (اسم، الأداة). لما تعمل `fit_transform()` الـ Pipeline كله بيشغّل الخطوات تلقائيًا بالترتيب: أول حاجة `SimpleImputer` (يملأ القيم الناقصة) وناتجها بيتمرر تلقائيًا لـ `StandardScaler` (يوحّد المقياس). الأسماء دي ("impute" و "standardize") اختيارية، هي labels عشان توضح كل خطوة بتعمل إيه.

نسخة أسهل: `make_pipeline` نفس الفكرة بالظبط لكن بيدّي أسماء تلقائية بدل ما تكتبهم:

```
from sklearn.pipeline import make_pipeline
num_pipeline = make_pipeline(SimpleImputer(strategy="median"), StandardScaler())
housing_num_prepared = num_pipeline.fit_transform(housing_num)    # one line runs all the steps
print(housing_num_prepared.shape, [name for name, _ in num_pipeline.steps])
```

Output (real run)

```
(16512, 8) ['simpleimputer', 'standardscaler']
```



الـ Pipeline يشغّل الخطوات بالترتيب ويمرّر ناتج كل خطوة للي بعدها.

ColumnTransformer

من كشكولك

المشكلة: الـ Pipelines اللي فاتت بتتطبّق على كل الأعمدة بنفس الشكل. لكن الداتا فيها أعمدة مختلفة: أعمدة رقمية (`median_income` وغيرها) و أعمدة نصية (`ocean_proximity`). الرقمية محتاجة `SimpleImputer` + `StandardScaler`، لكن النصية محتاجة `SimpleImputer` مختلفة (strategy بالأكثر تكرارًا) + `OneHotEncoder`. لازم أعالجهم منفصلين.

```
from sklearn.compose import ColumnTransformer
num_attribs = ["longitude", ..., "median_income"]     # numeric attributes
cat_attribs = ["ocean_proximity"]                     # categorical attributes

cat_pipeline = make_pipeline(
    SimpleImputer(strategy="most_frequent"),
    OneHotEncoder(handle_unknown="ignore"))

preprocessing = ColumnTransformer([
    ("num", num_pipeline, num_attribs),
    ("cat", cat_pipeline, cat_attribs),
])
```

من كشكولك

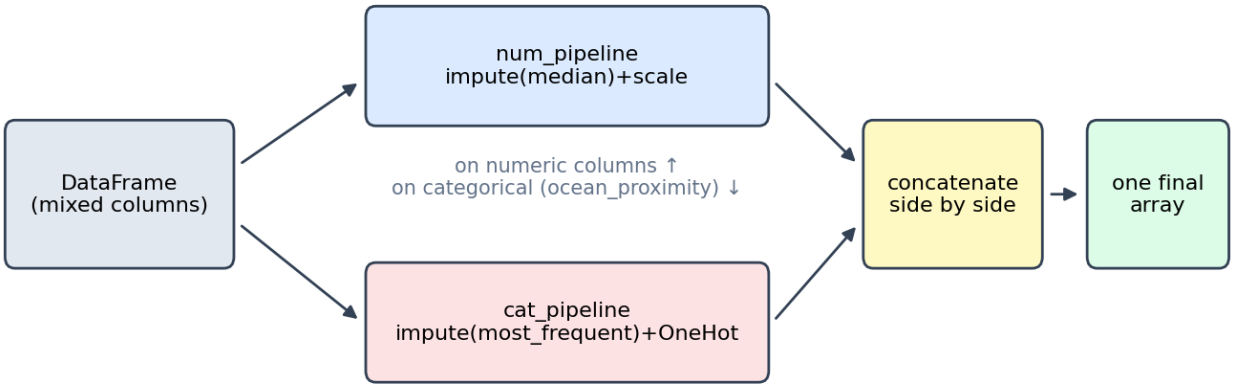
الفكرة: بتحدد لكل مجموعة أعمدة (قائمة أسماء) أي pipeline يتطبق عليها. الـ `ColumnTransformer` بيشغّل كل pipeline على الأعمدة المخصصة له، وفي الآخر بيلزق النتايج كلها جنب بعض في جدول واحد نهائي. هنا مش هنقدر نستخدم `median` لأن النص ملوش وسيط رياضي؛ فاستخدمنا الفئة الأكتر تكرارًا.

`make_column_selector`: اختيار الأعمدة تلقائيًا حسب النوع:

```
from sklearn.compose import make_column_selector, make_column_transformer
preprocessing = make_column_transformer(
    (num_pipeline, make_column_selector(dtype_include=np.number)),
    (cat_pipeline, make_column_selector(dtype_include=object)),
)
```

من كشكولك

بدل ما تكتب أسماء الأعمدة الرقمية والنصية يدويًا (وممكن أسماء الأعمدة أو الداتا تتغير)، بتختار تلقائيًا أي عمود رقمي أو نصي (أكثر مرونة).



ColumnTransformer: كل نوع أعمدة له pipeline، والنتيجة بتتجمّع في array واحد.

الـ Preprocessing الكامل

من كشكولك

الكود الآتي بيجمع كل حاجة اتعملت في المشروع في مكان واحد:

```
def column_ratio(X): return X[:, [0]] / X[:, [1]]
def ratio_name(function_transformer, feature_names_in): return ["ratio"]
def ratio_pipeline():
    return make_pipeline(SimpleImputer(strategy="median"),
                         FunctionTransformer(column_ratio, feature_names_out=ratio_name),
                         StandardScaler())
log_pipeline = make_pipeline(SimpleImputer(strategy="median"),
                             FunctionTransformer(np.log, feature_names_out="one-to-one"),
                             StandardScaler())
cluster_simil = ClusterSimilarity(n_clusters=10, gamma=1., random_state=42)
default_num_pipeline = make_pipeline(SimpleImputer(strategy="median"), StandardScaler())

preprocessing = ColumnTransformer([
    ("bedrooms", ratio_pipeline(), ["total_bedrooms", "total_rooms"]),
    ("rooms_per_house", ratio_pipeline(), ["total_rooms", "households"]),
    ("people_per_house", ratio_pipeline(), ["population", "households"]),
    ("log", log_pipeline, ["total_bedrooms", "total_rooms", "population", "households", "median_income"]),
    ("geo", cluster_simil, ["latitude", "longitude"]),
    ("cat", cat_pipeline, make_column_selector(dtype_include=object)),
], remainder=default_num_pipeline)     # any column not mentioned (housing_median_age) gets impute + scale

housing_prepared = preprocessing.fit_transform(housing)
print(housing_prepared.shape)
print(preprocessing.get_feature_names_out()[:5], "...", preprocessing.get_feature_names_out()[-3:])
```

Output (real run)

```
(16512, 24)
['bedrooms__ratio' 'rooms_per_house__ratio' 'people_per_house__ratio'
 'log__total_bedrooms' 'log__total_rooms'] ... ['cat__ocean_proximity_NEAR BAY' 'cat__ocean_proximity_NEAR
OCEAN'
 'remainder__housing_median_age']
```

من كشكولك

- `ratio_pipeline` : `FunctionTransformer` (القسمة) + `SimpleImputer` + `StandardScaler` .
- `cluster_simil` : بيحسب تشابه كل صف مع مراكز clusters (بالـ KMeans clustering + RBF) على latitude/ longitude.
- `remainder=default_num_pipeline` : أي عمود ماحدش ذكره في أي transformer بيتعمله الـ default (impute + scale).
- النتيجة: سطر واحد (`fit_transform`) بيطبّق كل خطوات المعالجة اللي قلنا عليها قبل كده على الداتا كلها.

إضافة

الناتج النهائي بعد الـ preprocessing هو `(16512, 24)` يعني 16,512 صف (80% من الـ 20,640) و 24 feature: 1 + 5 one-hot + 10 clusters + 5 log + 3 ratios (`housing_median_age`) = 24.

14 تدريب الموديلات الحقيقية

من كشكولك

أول لحظة فعلية بنتدرب فيها موديلات حقيقية بعد كل التحضيرات.

① Linear Regression

```
from sklearn.linear_model import LinearRegression
lin_reg = make_pipeline(preprocessing, LinearRegression())
lin_reg.fit(housing, housing_labels)                 # housing_labels = the real values the model must predict
housing_predictions = lin_reg.predict(housing)

from sklearn.metrics import root_mean_squared_error
lin_rmse = root_mean_squared_error(housing_labels, housing_predictions)
print(housing_predictions[:5].round(-2)); print(housing_labels.iloc[:5].values); print(lin_rmse)
```

Output (real run)

```
[ 70700. 288200. 186000. 189700. 270700.]
[ 72100. 279600.  82700. 112500. 238300.]
69240.0
```

من كشكولك

بتشوف جزء من `housing_predictions` وجزء من `housing_labels` : بعض التنبؤات غلط بمقدار كبير لكن باقي التنبؤات أقرب، يعني الموديل شغّال بس مش دقيق أوي. الدالة `root_mean_squared_error` بترجع RMSE مباشرة (يعني بتاخد الجذر التربيعي للـ MSE) بدل MSE.

الناتج بتاع RMSE كبير نسبيًا، يعني الموديل مش مضبوط، فهو Underfitting: الموديل بسيط جدًا مقارنة بتعقيد المشكلة، مش قادر يلتقط النمط الحقيقي للبيانات.

إضافة

من الكتاب: RMSE ≈ 68,688 دولار. عندي في التشغيل الحقيقي على الداتا دي: **69,240** (قريب؛ النسخة اللي اتحمّلت هنا مش مطابقة 100% لنسخة الكتاب)، في حين إن أغلب قيم `median_house_value` بين 120,000 و 265,000، فخطأ كبير. الكتاب بيدي 3 حلول للـ underfitting: (1) موديل أقوى، (2) features أحسن، (3) تقليل القيود (regularization). الموديل ده مش regularized فالحل 3 مستبعد، فنجرب موديل أعقد.

من كشكولك

Decision Tree ②

```
from sklearn.tree import DecisionTreeRegressor
tree_reg = make_pipeline(preprocessing, DecisionTreeRegressor(random_state=42))
tree_reg.fit(housing, housing_labels)
tree_rmse = root_mean_squared_error(housing_labels, tree_reg.predict(housing))
print(tree_rmse)
```

Output (real run)

```
0.0
```

من كشكولك

نفس الفكرة بس بموديل أقوى وأكتر تعقيدًا (هيتشرح في Ch6). لما أحسب RMSE بنفس كود `lin_rmse` هالقيه **0**. طبعًا ده شبه overfitting ألن السبب: إحنا بنختبر الموديل على نفس البيانات اللي اتدرب عليها، فأكيد حفظها وبيطلع نتيجة مثالية كاذبة (الخطأ صفر!). لازم نختبر الموديل على بيانات منشافهاش قبل كده.

المشكلة: مينفعش نستخدم الـ test set دلوقتي لأن القاعدة الذهبية بتقول ميتلمسش إلا لما تكون جاهز تطلق الموديل النهائي. فإحنا محتاجين نقيّم الموديل بشكل صادق من غير ما نجيب الـ test set. الحل: Cross-Validation.

Cross-Validation (k-fold)

من كشكولك

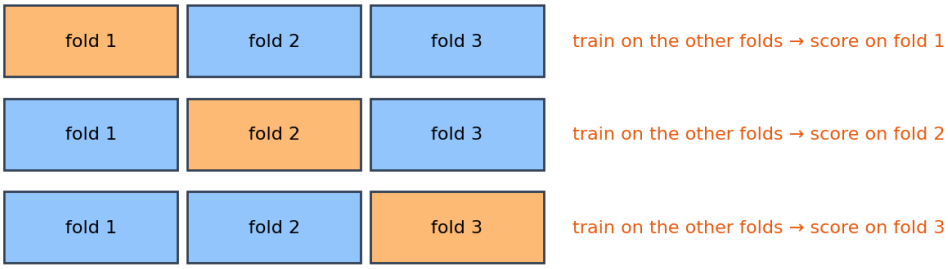
الفكرة: بدل ما نقسم البيانات مرة واحدة بس (Train/Validation)، الـ Cross-Validation بيقسّم الـ training set لـ 10 أجزاء متساوية (folds) ويعمل 10 مرات: (1) ناخد جزء واحد كـ "اختبار مؤقت" (Validation). (2) ندرّب الموديل على الـ 9 أجزاء التانية. (3) نقيس الأداء على الجزء المستبعد.

وتتكرر العملية دي 10 مرات، كل مرة بجزء مختلف كـ "اختبار"، وفي الآخر بترجعلك 10 نتايج RMSE مختلفة. **ليه ده أفضل؟** لأنه أشمل وأصدق من الاعتماد على تقسيمة واحدة ممكن تطلع محظوظة أو سيئة بالصدفة. وكمان تقدر تحسب الانحراف المعياري للـ 10 نتايج وده بيدّيك فكرة عن مدى ثبات أداء الموديل مش بس متوسطه.

```
from sklearn.model_selection import cross_val_score
tree_rmses = -cross_val_score(tree_reg, housing, housing_labels,
                              scoring="neg_root_mean_squared_error", cv=10)
print(tree_rmses.mean(), tree_rmses.std())
```

Output (real run)

```
67431.0  3437.1
```



فكرة الـ k-fold (الرسمة بـ 3 folds للتبسيط): كل جولة بتستبعد fold مختلف للتقييم.

من كشكولك

ملحوظة (الناقص): الـ `cross_val_score` بتتوقع دالة "منفعة" (utility function: كل ما زادت كل ما أحسن) عكس RMSE اللي هي دالة تكلفة (cost function: كل ما قلّت كل ما أحسن). عشان كده sklearn بترجّع القيمة بالسالب (`neg_root_mean_squared_error`)، وإحنا بنحط علامة سالب قدامها في الأول لقلبها.

النتيجة زي مثال الكتاب: ردّت كبيرة جدًا (فعلًا overfitting)، وده بالظبط سبب أهمية Cross-Validation: كشف الحقيقة اللي التقييم الساذج كان مخبيها.

إضافة

من الكتاب: متوسط RMSE للـ Decision Tree في الـ cross-validation ≈ 66,868. عندي: **67,431** (±3,437) ← يعني مش أحسن من الـ Linear Regression تقريبًا، وده بيأكد إن الـ 0 اللي طلع على الـ train كان overfitting.

من كشكولك

Random Forest ③

```
from sklearn.ensemble import RandomForestRegressor
forest_reg = make_pipeline(preprocessing, RandomForestRegressor(random_state=42))
forest_rmses = -cross_val_score(forest_reg, housing, housing_labels,
                                scoring="neg_root_mean_squared_error", cv=10)
print(forest_rmses.mean(), forest_rmses.std())
```

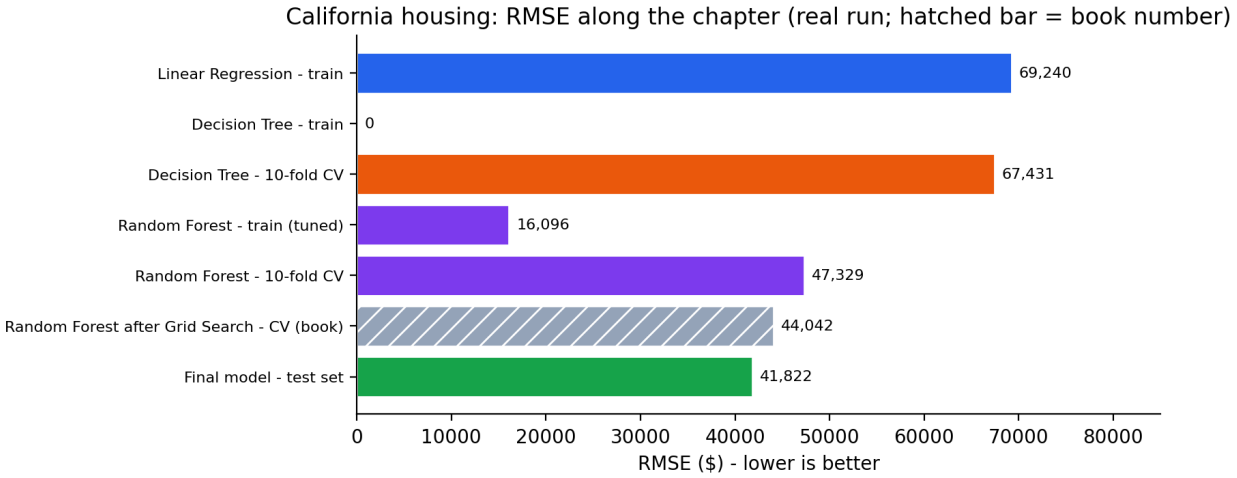
```
Output (real run)
47328.8  2330.4
```

من كشكولك

مفهوم Ensemble (تجميع الموديلات): الكتاب بيقدم فكرة الـ Random Forest، يعني موديل مركّب من موديلات كتير أصغر (هتاخد معظم التفاصيل في Ch7) لتلافي نفس النتايج وأفضل (بالنسبة لمثال الكتاب) overfitting لسه موجود بس مش بنفس الحدة (أقل).

إضافة

من الكتاب: متوسط RMSE للـ Random Forest في الـ cross-validation ≈ 47,019 (انحراف معياري ≈ 1,034). عندي: **47,329** (± 2,330). وده أحسن بكتير من الـ Tree. لكن RMSE على الـ training set نفسه (الموديل النهائي بعد الضبط) = **16,096**، وهو أقل بكتير من الـ CV ← يعني لسه في overfitting. حلول الـ overfitting حسب الكتاب: تبسيط الموديل، تقييده (regularization)، أو جمع داتا أكتر بكتير.



رحلة الأرقام في الشابتر (تشغيل حقيقي): لاحظ إن RMSE على الـ train ممكن يكون مضلل (0 للـ tree)، فالـ cross-validation هو اللي بيكشف الحقيقة.

15 ضبط الموديل (Fine-Tune)

من كشكولك

لنفترض إن عندك دلوقتي قائمة مختصرة من النماذج الواعدة، دلوقتي محتاجين نحسّنهم (يعني في مثال الكتاب كان أفضل اختيار موديل Random Forest، فدلوقتي هنشوف إزاي نحسّنه).

① Grid Search (البحث الشبكي)

من كشكولك

المشكلة: موديل مثل Random Forest عنده hyperparameters، والقيم الافتراضية بتاعتها مش بالضرورة الأفضل. ولو حاولت تجرّب كل القيم يدويًا هتاخد وقت طويل. **الحل:** `GridSearchCV`

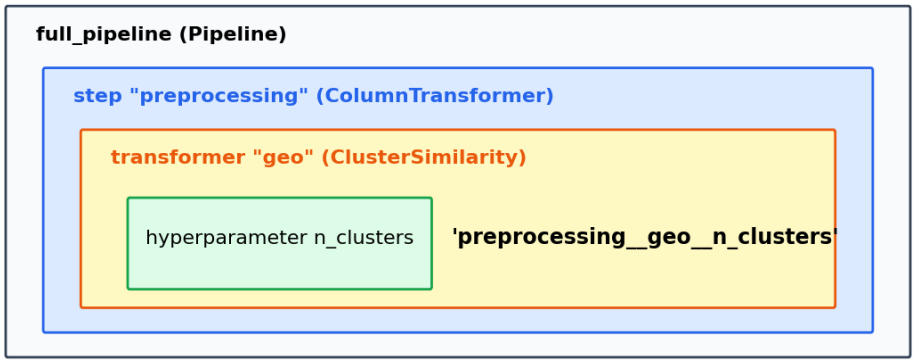
```
from sklearn.model_selection import GridSearchCV

full_pipeline = Pipeline([
    ("preprocessing", preprocessing),
    ("random_forest", RandomForestRegressor(random_state=42)),
])
param_grid = [
    {'preprocessing__geo__n_clusters': [5, 8, 10], 'random_forest__max_features': [4, 6, 8]},
    {'preprocessing__geo__n_clusters': [10, 15],   'random_forest__max_features': [6, 8, 10]},
]
grid_search = GridSearchCV(full_pipeline, param_grid, cv=3, scoring='neg_root_mean_squared_error')
grid_search.fit(housing, housing_labels)
```

من كشكولك

الفكرة: بتديله قائمة قيم لكل hyperparameter وهو بيجرّب كل التوليفات الممكنة (كل الاحتمالات)، وبيقيّم كل توليفة باستخدام Cross-Validation ويقولك إنهي توليفة أحسن.

الجزء المهم: الاسم `'preprocessing__geo__n_clusters'` مش اسم عشوائي، هو مسار لـ hyperparameter عميق جوه هيكل متداخل. خلينا نفصّله: `preprocessing` اسم الخطوة في `full_pipeline` (اللي هو الـ ColumnTransformer)، `geo` اسم الـ transformer جوه الـ ColumnTransformer (اللي هو `cluster_simil`)، و`n_clusters` الـ hyperparameter جوه الـ `geo`. sklearn بيفصل السلسلة بـ underscore مزدوج (`__`) وينزل درجة جوه الهيكل المتداخل.



اسم الـ hyperparameter = مسار جوه الهيكل المتداخل، مفصول بـ `__`

من كشكولك

الحساب الحسابي: القاموس الأول: 3 × 3 = 9 توليفات، الثاني: 2 × 3 = 6 توليفات. الإجمالي 15 توليفة. وكل توليفة بتتدرّب 3 مرات (`cv=3`)، فالإجمالي الكلي = 15 × 3 = **45 عملية تدريب**. ده بيوضح ليه Grid Search بياخد وقت طويل: كل توليفة بتتكرر بعدد الـ folds.

النتيجة: `grid_search.best_params_` هي أفضل توليفة، والنواتج. لو طلعت أعلى قيمة من اللي إنت مدّيهم في آخر المدى، يبقى لازم توسّع النطاق لأنك مش متأكد إن ده كان أفضل المطلق، ممكن تجرب أكتر وتكرر.

إضافة

- من الكتاب: أفضل توليفة: `n_clusters=15` و `max_features=6`، وأفضل RMSE (cross-val) ≈ **44,042** (أحسن من 47,019 اللي كانت بالقيم الافتراضية). (النص في الكتاب بيقول `max_features=8` لكن الـ output والجدول بيقولوا 6.)
- لأن 15 هي أعلى قيمة اتجرّبت لـ `n_clusters` فالكتاب بينبه إنك تعيد البحث بقيم أعلى.
- `GridSearchCV` بيعمل `refit=True` بشكل افتراضي: بعد ما يلاقي أحسن توليفة بيدرّبها تاني على الـ training set كله. وتقدر تقرا كل النتايج من `grid_search.cv_results_` (حطها في DataFrame).
- (الجدول ده ما اتشغلش هنا: 45 تدريب على Random Forest ثقيل على جهاز بمعالج واحد، فأرقامه من الكتاب. بس أخدنا أفضل توليفة منه وشغّلناها فعلًا في القسم 16.)

② Randomized Search (بديل أذكى لما يكون فيه خيارات كتيرة)

من كشكولك

المشكلة في Grid Search: لو عندك 6 hyperparameters وكل واحد له 10 قيم مختلفة، هتحتاج مليون تدريب (10^6)، ودي غير عملية خالص.

```
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import randint

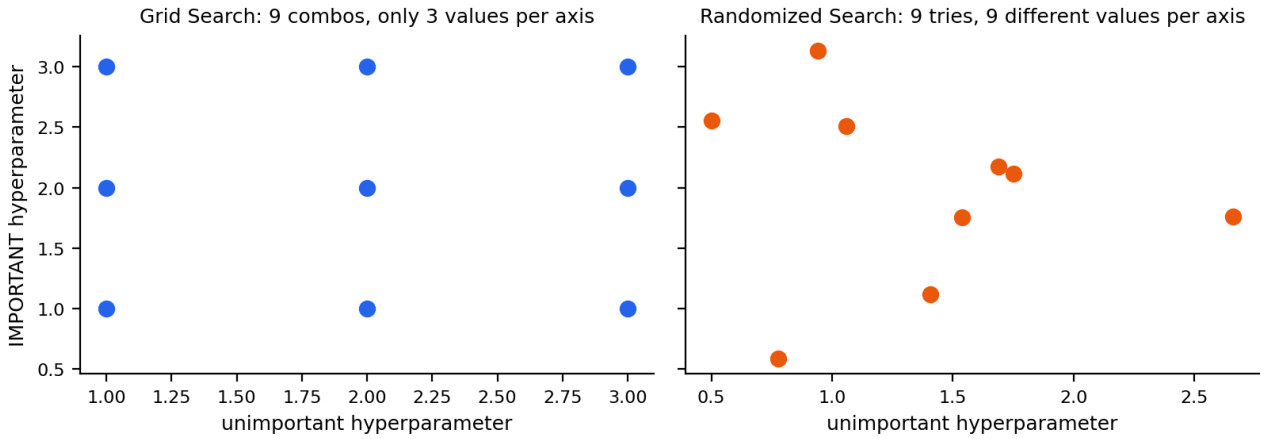
param_distribs = {'preprocessing__geo__n_clusters': randint(low=3, high=50),
                  'random_forest__max_features': randint(low=2, high=20)}
rnd_search = RandomizedSearchCV(full_pipeline, param_distributions=param_distribs,
                                n_iter=10, cv=3, scoring='neg_root_mean_squared_error', random_state=42)
rnd_search.fit(housing, housing_labels)
```

من كشكولك

الفرق الجوهري: بدل تجربة كل التوليفات، بيعمل عدد محاولات ثابت (`n_iter=10`)، وفي كل محاولة بيختار قيمة عشوائية لكل hyperparameter من `randint(low, high)` (توزيع بين قيمتين)، مش كل التوليفات، بس عينة عشوائية منها.

ليه أفضل؟

1. **مرونة أكبر:** مع `n_iter=1000` بيستكشف قيم مختلفة لكل hyperparameter، مش بس القيم القليلة اللي كتبتها يدويًا في Grid Search (يعني 1000 قيمة مختلفة لكل hyperparameter).
2. **توفير الوقت:** لو hyperparameter معيّن مش فارق (مش مؤثر)، Grid Search بيضاعف وقت التدريب، لكن Randomized Search عدد المحاولات ثابت.
3. **تحكم كامل في الوقت:** بدل "مليون تدريب إجباري"، إنت بتحدد عدد المحاولات (`n_iter`) حسب الميزانية، وتقدر تتحكم بحجم مساحة البحث.



نفس عدد المحاولات (9): الـ Grid بيجرّب 3 قيم بس للـ hyperparameter المهم، والـ Random بيجرّب 9 قيم مختلفة.

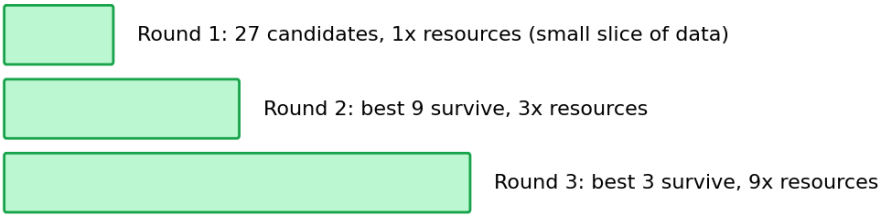
③ Halving Random Search / Halving Grid Search

من كشكولك

ملاحظة سريعة: `HalvingRandomSearchCV` و `HalvingGridSearchCV` : خيارات إضافية أكتر كفاءة وأسرع. الفكرة: طرق البحث دي بتدور على أفضل hyperparameters بكفاءة أعلى من البحث العادي.

1. يبدأوا بعدد كبير من الـ candidates (توليفات hyperparameters).
2. يجرّبوا كل الـ candidates بموارد قليلة (مثل جزء صغير من بيانات التدريب).
3. يقيّموا النتائج باستخدام Cross-Validation.
4. يحتفظوا فقط بأفضل الـ candidates.
5. في الجولة التالية يعطوا الـ candidates الباقية موارد أكبر.
6. يكرروا العملية لحد ما يوصلوا لأفضل candidates باستخدام موارد كاملة.

الهدف: توفير الوقت والـ Computational resources، أو القدرة على تجربة عدد أكبر من الـ combinations في نفس الوقت.



فكرة الـ Halving: ابدأ بكتير وبموارد قليلة، وكمّل بالأحسن بموارد أكبر.

إضافة

الـ Halving classes لسه "experimental" في sklearn، فلازم تفعّلها الأول بـ `from sklearn.experimental import enable_halving_search_cv` قبل ما تعمل import ليهم.

④ Ensemble Methods

من كشكولك

في طريقة أخرى لتحسين أداء النظام: محاولة دمج أفضل موديلات مع بعض (زي Random Forest + KNN، وتاخد متوسط تنبؤاتهم). فكرة ensemble بس دي مستوى "دمج موديلات كاملة مختلفة" مش "دمج أشجار قرار" زي Random Forest (هتتشرح في Ch7).

⑤ تحليل أفضل الموديلات (Feature Importances)

من كشكولك

بعد ما `rnd_search` يخلص، فيه عندي: `best_params_` (أفضل hyperparameters) و `best_estimator_` (أفضل موديل). `final_model` هو الـ Pipeline كامل (ضمنه خطوة `"random_forest"`)، و `final_model["random_forest"]` بيجيب الـ `RandomForestRegressor` اللي جوه الـ Pipeline. `feature_importances_` : بيقولك كل feature كانت مهمة قد إيه في قرارات الـ Random Forest (كل ما الرقم أكبر كان لها وزن أكبر في الموديل).

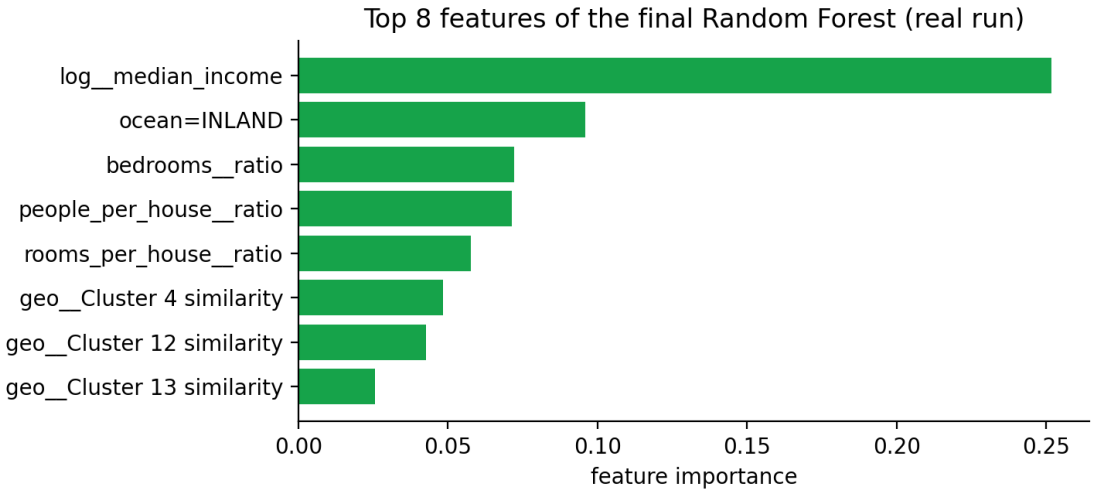
```
final_model = rnd_search.best_estimator_
feature_importances = final_model["random_forest"].feature_importances_
sorted(zip(feature_importances.round(3), final_model["preprocessing"].get_feature_names_out()),
reverse=True)[:8]
```

Output (real run)

```
0.252  log__median_income
0.096  cat__ocean_proximity_INLAND
0.072  bedrooms__ratio
0.071  people_per_house__ratio
0.058  rooms_per_house__ratio
0.048  geo__Cluster 4 similarity
0.043  geo__Cluster 12 similarity
0.026  geo__Cluster 13 similarity
```

إضافة

(الأرقام فوق من الموديل اللي دربناه بأفضل توليفة للكتاب.) الـ zip + `get_feature_names_out()` بيربط كل رقم أهمية باسم الـ feature ويعرضهم من الأكبر للأقل (`reverse=True`). ملحوظة: `feature_importances_` تعني إن الـ feature "مهمة" للموديل بشكل مبدئي، يعني Random Forest استخدمها بدرجة كبيرة في تقليل الخطأ أثناء بناء الأشجار. وتلميح من الكتاب: `SelectFromModel` بيدرّب موديل (غالبًا Random Forest)، يبص على `feature_importances_` ويشيل الـ features الأقل فايدة تلقائيًا. وكمان تحلل الأخطاء بنفسك: إيه الحالات اللي الموديل بيغلط فيها وليه (إضافة feature، تنضيف outliers، إلخ).



أهم الـ features في الموديل النهائي (تشغيل حقيقي): الدخل (`median_income`) والموقع (`INLAND` و`log__median_income`) في الصدارة، وده نفس اللي توقعناه من الـ correlation.

انتبه / تصحيح

تحذير مهم: لازم تتأكد إن الموديل بيشتغل كويس على كل الفئات الفرعية (زي المناطق الفقيرة والغنية)، مش بس على المتوسط العام. لو الموديل ضعيف على فئة معينة (زي مناطق فقيرة مثلًا) وانطلق Production وهو بيدي نتايج سيئة لفئة كاملة من الناس، ده ممكن يكون خطر أخلاقي. ده مبدأ عام مهم في أي مشروع ML حقيقي.

16 التقييم النهائي وفاصل الثقة (Confidence Interval)

من كشكولك

هنا نلمس أخيرًا الـ `test_set` ونشوف النواتج وما إلى ذلك. بعد Cross-Validation هنعمل Confidence Interval لقياس مدى الثقة في الرقم، عشان نستخدم الأدوات الإحصائية.

```
X_test = strat_test_set.drop("median_house_value", axis=1)
y_test = strat_test_set["median_house_value"].copy()
final_predictions = final_model.predict(X_test)
final_rmse = root_mean_squared_error(y_test, final_predictions)

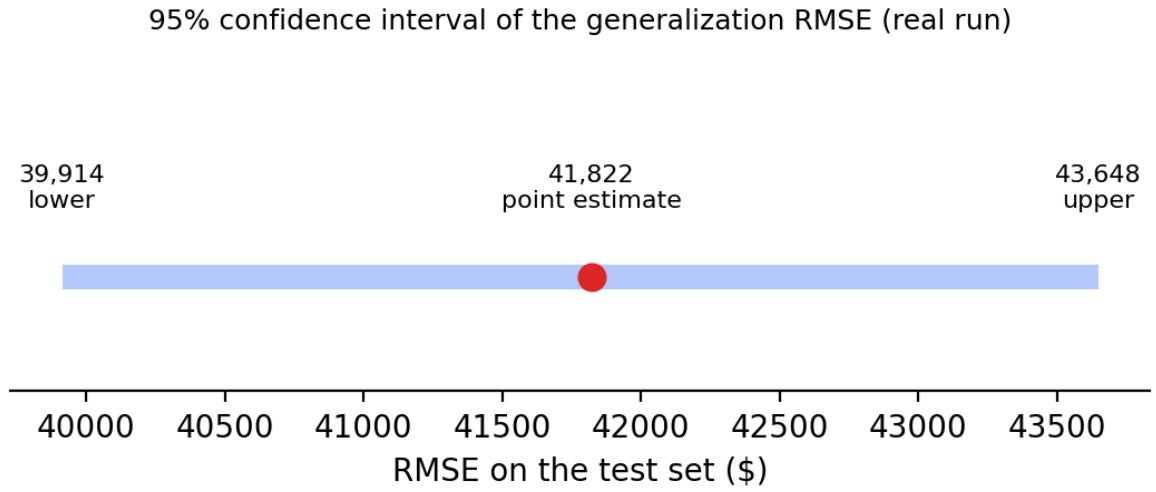
from scipy import stats
confidence = 0.95
squared_errors = (final_predictions - y_test) ** 2
np.sqrt(stats.t.interval(confidence, len(squared_errors) - 1,
                         loc=squared_errors.mean(), scale=stats.sem(squared_errors)))
```

Output (real run)

```
final RMSE = 41822.3
[39913.6  43647.5]   # 95% confidence interval
```

من كشكولك

- `confidence = 0.95` : يعني بنحسب نطاق ثقة بمستوى 95%.
- `squared_errors` : هنا بيحسب خطأ كل prediction (التنبؤ) بالنسبة للقيمة الحقيقية، مربّع.
- `len(squared_errors) - 1` = degrees of freedom = n − 1: لو عندك n من الأخطاء، فدرجات الحرية n − 1. وده مطلوب في حساب الـ t-distribution (الدالة محتاجة تعرف عدد درجات الحرية المستخدم في حساب الـ Confidence Interval).
- `loc = squared_errors.mean()` : مركز الـ Confidence Interval، يعني متوسط الـ squared errors (يتحسب MSE).
- `scale = stats.sem(squared_errors)` : Standard error of the mean: مقدار عدم اليقين في تقدير متوسط الـ squared errors.
- `np.sqrt` بتحوّل النتيجة من MSE لـ RMSE (أول رقم هو الحد الأدنى والتاني الحد الأقصى).



تشغيل حقيقي: RMSE على الـ test set = 41,822، وفاصل ثقة 95% من 39,914 إلى 43,648. (الكتاب: 41,424 وفاصل 39,275 إلى 43,467)

إضافة

الـ RMSE على الـ test (41,822) أقل من الـ CV (44,042 في الكتاب)، وده طبيعي لأن كل fold في الـ CV بيتدرّب على جزء بس من الـ training set، أما الموديل النهائي فاتدرّب عليه كله. وده معناه إن الموديل بيعمّم كويس. ولاحظ الفرق بين RMSE على الـ train (16,096) وعلى الـ test: الفجوة دي هي الـ overfitting اللي كلمنا عنه.

انتبه / تصحيح

بعد ما تقيّم على الـ test set متحاولش "تظبّط" الموديل عشان الرقم يتحسن؛ التحسينات دي مش هتتعمم على داتا جديدة. وده معناه إن الـ test set اسْتُخدم خلاص وبقى زي الـ validation.

17 الخطوتان 7 و 8 (مكمّلة من عندي)

إضافة

Present your solution: وثّق اللي اتعلمته: إيه اللي نجح وإيه اللي فشل، الافتراضات، والقيود (limitations)، وقدّم النتايج بشكل واضح (رسومات أو جمل سهلة بدل أرقام بس).

Launch: جهّز الكود للـ production (تنضيف، tests، documentation) ثم انشر الموديل. أبسط طريقة: احفظه وانقله:

```
import joblib
joblib.dump(final_model, "my_california_housing_model.pkl")
final_model_reloaded = joblib.load("my_california_housing_model.pkl")   # custom classes
(ClusterSimilarity...) must be importable
```

تلميح من الكتاب: احفظ كل موديل بتجربه (ودرجات الـ cross-validation والتنبؤات) عشان تقارن بينهم بسهولة بعدين.

Monitor & Maintain: الموديل بيفقد جودته مع الوقت لأن العالم بيتغيّر (داتا جديدة بتختلف عن داتا التدريب)، فلازم تراقب أداؤه باستمرار، وتعيد تدريبه بداتا جديدة بانتظام، وتأتمت العملية دي (MLOps) وتحتفظ بنسخ من الداتا والموديلات.

18 أسئلة مراجعة سريعة

جاوب الأول في دماغك، وبعدين شوف الإجابات في الصفحة اللي بعدها.

1. إيه الفرق بين RMSE و MAE، ومتى أفضّل MAE؟
2. لو $R^2 = 0.8$، هل ده معناه إن 80% من التنبؤات صح؟
3. إيه مشكلة التقسيم العشوائي لما الداتا بتتحدّث؟
4. إزاي الـ hash (crc32) بيحل المشكلة دي؟
5. ليه رقم الصف (index) مش ID كويس؟
6. إيه فايدة Stratified Sampling؟ وإزاي طبقناه على income_median؟
7. ليه نعمل fit على الـ training set بس، ونعمل transform على الباقي؟
8. ليه OneHotEncoder أحسن من OrdinalEncoder للفئات اللي مالهاش ترتيب؟
9. إيه الـ Sparse matrix ولما نحتاجها؟
10. إيه الفرق بين Min-Max و Standardization؟ وأيهم أحسن مع outlier؟
11. إيه المشكلة في feature heavy-tail، وإيه الحل؟
12. إمتى أستخدم Bucketizing مع OneHot؟
13. إيه الـ RBF similarity feature؟
14. ليه استخدمنا FunctionTransformer؟ وTransformedTargetRegressor؟
15. إيه الفرق بين Pipeline و ColumnTransformer؟
16. ليه RMSE = 0 للـ Decision Tree على الـ training علامة سيئة؟ وإزاي نكشفها؟
17. ليه cross_val_score بترجع قيم سالبة؟
18. Grid Search: 15 توليفة و cv=3، كام تدريب؟ وليه Randomized أفضل؟
19. إيه معنى 'preprocessing__geo__n_clusters'؟
20. إيه الغرض من Confidence Interval على الـ test set؟

19 إجابات أسئلة المراجعة

1. RMSE بيستخدم $\ell 2$ (تربيع) فبيتأثر جدًا بالـ outliers، وMAE بيستخدم $\ell 1$ (قيم مطلقة) فأهدى. لو الـ outliers كتير أفضّل MAE، ولو نادرة RMSE ممتاز.
2. لأ. معناه إن الموديل بيفسر حوالي 80% من التباين في الـ target مقارنة بالتنبؤ بالمتوسط بس. مش Accuracy.
3. كل مرة الترتيب بيتغير، فصفوف ممكن تنتقل من train لـ test، فالموديل يكون "شاف" بيانات الـ test (data leakage).
4. بيحوّل ID ثابت لرقم ثابت، فنفس الصف دايمًا يروح لنفس المجموعة (لو hash < test_ratio × 2^32 يبقى test).
5. لأنه بيتغير لو اتمسح صف من النص؛ لازم الـ ID يتكوّن من حاجة ثابتة وفريدة.
6. بيحافظ على نسبة كل فئة في الـ train/test. حوّلنا الدخل لفئات بـ pd.cut ثم استخدمنا stratify=income_cat، وبعدها مسحنا العمود.
7. عشان الإحصائيات (وسيط/متوسط) تتعلم من التدريب بس، ولو اتحسبت من الـ test يحصل leakage.
8. Ordinal بيديها أرقام فالموديل يفترض قرب/بعد مش حقيقي. One-hot بيعمل عمود لكل فئة فكلها متساوية ومستقلة.
9. تخزين مواقع الـ 1 بس بدل كل الأصفار؛ بتوفر ذاكرة جدًا لما الفئات كتيرة. toarray() أو sparse_output=False لو عايز array عادي.
10. Min-Max بيحط القيم في مدى ثابت فالـ outlier بيضغط الباقي. Standardization بيخلّي المتوسط 0 والانحراف 1 وأقل تأثرًا بالـ outlier.
11. الموديل بيتعلم منها بصعوبة. الحل: اللوغاريتم (أو الجذر/ bucketizing) عشان يقرب التوزيع من الجرس.
12. لما الـ feature متعدد القمم (multimodal) فكل صندوق يتعامل كفئة منفصلة.
13. feature بتقيس القرب من قيمة ثابتة (مثلًا عمر 35): $\exp(-\gamma(x-35)^2)$؛ = 1 عند 35 وبتقل كل ما ابتعدنا.
14. الأولى بتحوّل أي function لـ transformer يتحط في pipeline. التانية بتحوّل الـ target وترجع التنبؤات لقيمتها الحقيقية تلقائيًا.
15. Pipeline بيشغّل خطوات بالترتيب. ColumnTransformer بيطبّق pipeline مختلف على كل مجموعة أعمدة ويجمع الناتج.
16. overfitting: الموديل حفظ الداتا. نكشفها بـ cross-validation (≈ 66-67 ألف في المثال).
17. لأنها بتتوقع utility function (الأكبر أحسن)، فـ RMSE بيتحوّل لـ neg_root_mean_squared_error ونقلبه بعلامة سالب.
18. 45 تدريب. Randomized بيجرّب عدد ثابت (n_iter) بقيم عشوائية، فمرن أكتر ومتحكم في الوقت.
19. مسار: خطوة preprocessing ثم transformer اسمه geo ثم hyperparameter اسمه n_clusters، مفصولين بـ __.
20. نعرف مدى دقة تقدير الـ RMSE (مثلًا من X إلى Y بثقة 95%) مش بس رقم واحد.